Sprint 16 · Reto «Entrega final: evaluar, demostrar y documentar»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. El cliente (el docente) va a **recibir vuestro trabajo como si fuera un entregable real**. Quiere poder **clonar el repositorio y seguir el README**, **ver funcionar el demostrador**, **leer una memoria de 10–15 páginas** y **entender los riesgos** antes de decidir si adopta vuestro modelo. Para eso tenéis que: evaluar a fondo el modelo candidato del Sprint 15 (una sola vez en el test), buscar sesgos por subgrupos y **aplicar y medir** una medida para reducirlos, cerrar el registro de riesgos y clasificar el sistema según el RGPD y el AI Act, construir un demostrador que no se rompa con entradas erróneas, que avise de la incertidumbre y que tenga **plan B en vídeo**, y documentarlo todo (model card, README y memoria técnica).

Cómo se trabaja. Este es el sprint de cierre: no hay una plantilla de código que rellenar (salvo el demostrador), sino **vuestro proyecto de los sprints 14 y 15**, que ahora hay que terminar y dejar listo para que otra persona lo use. Os damos plantillas para el demostrador, la model card, la memoria y la lista de congelación. **Todo lo que haya en la rama principal del repositorio a las 14:30 del 10/12 es lo que se evalúa**; pasada esa hora, se congela. Hay tres días de clase y, **entre el día 1 y el día 2, dos días sin clase**: buen momento para avanzar la memoria. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Vuestro proyecto (repositorio privado `ifcd107-equipo-…`)	El modelo candidato del Sprint 15, su registro de experimentos, los datos y el registro de riesgos de los sprints 14 y 15	Vuestro repositorio de GitHub
Cuaderno del S13 `S13_02`	Los gráficos de la evaluación (matriz de confusión, curvas, importancia de variables, SHAP o Grad-CAM) para adaptarlos	
Presentación S16.1 y **apuntes del sprint**	Qué es un demostrador, cómo se documenta y, con ejemplos de números, cómo se miden los sesgos y su mitigación	
Cuaderno del sprint `S16_01`	Práctica del demostrador: guardar el modelo y sus metadatos, separar la lógica, pruebas, Streamlit, widgets y API	
La carpeta del demostrador `demostrador/`	`logica.py`, `app.py`, `api.py` (opcional), `tests/test_logica.py`, `requirements.txt` y `README_demostrador.md`. Es un ejemplo completo con el modelo de TurisData: vosotros lo adaptáis a **vuestro** proyecto. La carpeta `modelo/` no viene hecha: la crea el cuaderno `S16_01` o vuestro script de entrenamiento	
Python y librerías (`scikit-learn`, `streamlit`, `pytest`…)	Ejecutar el cuaderno de evaluación, lanzar el demostrador y las pruebas	
Model card	Ficha técnica del modelo de máximo 2 páginas	
Esqueleto de la memoria	Secciones, presupuesto de páginas y mapa de requisitos R1–R11	
Lista de congelación	Repasar todo antes de las 14:30	
Coevaluación individual (anexo B)	Valorar el trabajo de cada persona del equipo. Es **confidencial**: va al docente, no al repositorio	
Cómo entregar en GitHub	Subir los ficheros a `sprint-16/`	
Autoevaluación y coevaluación	Cada persona rellena su hoja y la sube	

Recurso	Para qué lo usaréis	Dónde está
Un asistente de IA (opcional)	Entender sesgos, errores de código o revisar la redacción	

3 · Guía paso a paso

Son ocho pasos, de **525 minutos en total** repartidos en tres días. Las jornadas de proyecto duran **6 horas completas (250 minutos de trabajo al día)**: los pasos de esta ficha ocupan unos 200 y los **50 minutos restantes de cada día son «subir de nivel»** (pasar del mínimo al estándar con la rúbrica en la mano). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. El día 1 empieza con 10 minutos de planning, la píldora de teoría y la práctica del demostrador (`S16_01`), que no se cuentan en los 525. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para los dos días sin clase.

Paso 1 · Evaluar el modelo en el test y buscar dónde falla (día 1, 40 minutos)

1. Evaluar el modelo candidato del Sprint 15 en el test, **una sola vez**, y anotar las métricas con su incertidumbre.
2. Adaptar los gráficos de `S13_02` y guardar todo en `notebooks/04_evaluacion.ipynb`.
3. Calcular la métrica por subgrupos y sacar la lista de los errores más frecuentes.

En sencillo. El **test** es el examen final del modelo: datos que apartasteis al principio y que nunca ha visto. Es como el examen de conducir: si lo repetís hasta aprobar, la nota ya no mide nada. Por eso se hace **una vez** y se apunta el resultado, sea el que sea. Pero una nota global no basta: un modelo puede acertar «el 83 %» y fallar mucho más con un grupo (un canal de venta, un país). Ese fallo desigual se llama **sesgo**, y este paso sirve para encontrarlo y para mirar ejemplos reales de errores. No hay que programar de cero: partís de una plantilla que ya funciona (`S13_02`) y cambiáis lo marcado como `# >>> EDITA`. Los códigos de abajo funcionan tal cual con los datos de TurisData: ejecutadlos primero así, para ver qué sale, y después ponedlos con los datos de **vuestro** proyecto.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Test	Datos apartados desde el principio para el examen final. Se usan una sola vez.
Métrica	Un número que resume cómo lo hace el modelo. **AUC**: 0,5 es azar y 1 es perfecto. **Recall**: de las cancelaciones reales, cuántas detecta. **Precisión**: de las que avisa, cuántas ocurren de verdad.
Umbral (`UMBRAL`)	La probabilidad a partir de la cual el modelo «avisa». En `S13_02` vale 0,35.
Falsa alarma / no detectada	Aviso de cancelación que luego no ocurrió / cancelación que el modelo no vio venir.
Intervalo del 95 %	El tramo (de A a B) donde probablemente está la cifra real. Se calcula barajando el test 300 veces (*remuestreo* o *bootstrap*).
Subgrupo	Parte de los datos que se mide por separado: un canal, un país, un tipo de foto…
Sesgo	Que el modelo funcione peor para un grupo que para otros.
Título-conclusión	Título de gráfico que dice lo que se ve («En el canal directo detectamos solo el 62 %»), no lo que es («Recall por canal»).

Cómo se hace

1. **Crear el cuaderno (5 minutos).** Abrid `S13_02` en Colab, *Archivo → Guardar una copia en Drive* y llamadla `04_evaluacion`. Pulsad *Compartir* y añadid al equipo como **Editor**. Una persona escribe y las demás miran; cambiad de turno. Pulsad **▶ por orden** en las celdas hasta «2. Cálculos que alimentan el panel». Con TurisData sale `AUC en prueba: 0.744 · reservas de prueba: 1500`: así sabéis que funciona.

2. **Poner vuestro modelo (10 minutos).** En «1. Datos y modelo» cambiad solo lo marcado `# >>> EDITA`: leed **vuestros** datos y repetid la **misma partición** del Sprint 15 (copiad la línea `train_test_split` de `03_modelos.ipynb`: misma `random_state`, mismo `test_size`). En vez de entrenar, cargad el modelo guardado:

```
import joblib
modelo = joblib.load("modelo_candidato.joblib")
prob = modelo.predict_proba(X_test)[:, 1]
```

   Qué hace cada línea:

   - `joblib.load("modelo_candidato.joblib")`: lee el modelo ya entrenado (con su preprocesado). Poned el nombre de **vuestro** fichero y arrastradlo al panel de archivos de Colab (icono , a la izquierda) desde `sprint-15/` de vuestro repositorio. Si Colab se reinicia, hay que subirlo otra vez.
   - `modelo.predict_proba(X_test)[:, 1]`: la probabilidad de «cancela» de cada reserva del test. **Se llama `prob`**: las celdas siguientes usan ese nombre, y también `X_test`, `y_test`, `pred`, `auc` y `UMBRAL`. No los renombréis.

   Si vuestro proyecto no es una tabla (imágenes, series, asistente), la métrica cambia (F1, MAE, aciertos…), pero la estructura es la misma: una cifra general y la misma cifra por grupo. Consultadlo en la tutoría.

3. **Evaluar en el test, una vez (8 minutos).** En una celda nueva (**+ Código**):

```
from sklearn.metrics import recall_score, precision_score

print("Reservas de prueba:", len(y_test))
print(f"AUC {auc:.3f} · recall {recall_score(y_test, pred):.3f} · precisión {precision_score(y_test, pred):.3f} (umbral
{UMBRAL})")

rng = np.random.default_rng(SEMILLA)
real = y_test.to_numpy()
aucs, recalls = [], []
for _ in range(300):
    idx = rng.integers(0, len(real), len(real))
    if len(np.unique(real[idx])) < 2:
        continue
    aucs.append(roc_auc_score(real[idx], prob[idx]))
    recalls.append(recall_score(real[idx], pred[idx]))
for nombre, valores in (("AUC", aucs), ("Recall", recalls)):
    inf, sup = np.percentile(valores, [2.5, 97.5])
    print(f"{nombre}: intervalo 95 % entre {inf:.3f} y {sup:.3f}")
```

   Qué hace cada parte:

   - Las dos primeras líneas con `print` escriben las tres métricas. `auc` y `pred` (1 si `prob` ≥ `UMBRAL`, 0 si no) ya los calculó la celda «2. Cálculos». La `f` antes de las comillas deja poner variables entre llaves, y `:.3f` las escribe con 3 decimales.
   - El bucle `for` es el remuestreo de `S10_04`: 300 veces elige al azar `len(real)` reservas del test (pueden repetirse) y recalcula la métrica. **No vuelve a pasar el modelo por el test**: usa las mismas predicciones, así que sigue siendo «una sola vez».
   - `if len(np.unique(real[idx])) < 2: continue` salta los barajados con una sola clase, donde el AUC no se puede calcular.
   - `np.percentile(valores, [2.5, 97.5])` se queda con el 95 % central de las 300 cifras: es el intervalo.

   Copiad las cifras a una celda de texto («AUC 0,744, entre 0,719 y 0,767»). No cambiéis el modelo ni `UMBRAL` para mejorarlas.

4. **Gráficos con título-conclusión (7 minutos).** Pulsad ▶ en la celda «3. El panel de una página» (dibuja 4 gráficos) y cambiad los títulos marcados `# >>> EDITA` para que digan lo que muestran **vuestras** cifras. En TurisData: «Detectamos el 83 % de las cancelaciones, con un 49 % de falsas alarmas»; «Lo que más pesa en una cancelación es la antelación» (la variable líder sale de `importancias.head(6)`); y el del dinero, «Con el modelo perdemos 16,590 € menos en 1500 reservas».

5. **Métrica por subgrupos (8 minutos).** Pegad esta función (se queda en el cuaderno: la usaréis también en el paso 3) y usadla con la columna que queráis:

```
def medir_subgrupos(real, pred, grupo):
    filas = []
    for valor in sorted(set(grupo)):
        quien = grupo == valor
        r, p = real[quien], pred[quien]
        cancelan = int((r == 1).sum())
        detectadas = int(((r == 1) & (p == 1)).sum())
        avisadas = int((p == 1).sum())
        recall = detectadas / cancelan
        precision = detectadas / avisadas if avisadas else float("nan")
        margen = 1.96 * np.sqrt(recall * (1 - recall) / cancelan)
        filas.append({"subgrupo": valor, "reservas": int(quien.sum()), "cancelan": cancelan, "detectadas": detectadas,
                      "recall": round(recall, 3), "±": round(margen, 3), "precisión": round(precision, 3)})
    return pd.DataFrame(filas)

grupo = X_test["canal"].fillna("sin dato").to_numpy()
por_canal = medir_subgrupos(real, pred, grupo)
print(por_canal.to_string(index=False))

for columna in categoricas:
    tabla = medir_subgrupos(real, pred, X_test[columna].fillna("sin dato").to_numpy())
    peor = tabla.sort_values("recall").iloc[0]
    print(f"{columna}: el peor subgrupo es {peor['subgrupo']} (recall {peor['recall']:.3f}, {peor['cancelan']}
cancelaciones)")
```

   Qué hace cada parte:

   - `def medir_subgrupos(real, pred, grupo)`: una receta que recibe lo ocurrido (`real`), lo avisado (`pred`) y a qué grupo pertenece cada reserva (`grupo`). Por cada valor del grupo cuenta cuántas reservas cancelan, cuántas detecta y calcula el recall y la precisión.
   - `margen = 1.96 * np.sqrt(…)`: la columna ±. Es el margen del intervalo del 95 % de una proporción (el de los intervalos de confianza de S05): con pocos casos sale grande y avisa de que la cifra es poco fiable.
   - `X_test["canal"].fillna("sin dato")`: elige la columna `canal` y rellena los huecos con «sin dato» (un hueco hace fallar el orden; y **los datos que faltan también son un subgrupo**).
   - `for columna in categoricas`: repite lo mismo con todas las columnas de categorías (`categoricas` la creó S13_02) y escribe el peor subgrupo de cada una. Elegid el que tenga **un recall bajo y bastantes casos**.

6. **Gráfico de subgrupos (4 minutos).** Una barra por grupo, con su ±:

```
t = por_canal.sort_values("recall")
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.barh(t["subgrupo"], t["recall"], xerr=t["±"], color="tab:blue", capsize=4)
ax.axvline(recall_score(real, pred), color="gray", linestyle="--", label="Global")
ax.set_xlabel("Recall: cancelaciones detectadas (± margen)")
ax.set_title("En el canal directo detectamos solo el 62 % de las cancelaciones")
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

   `barh` dibuja barras horizontales y `xerr` les pone el ±; `axvline` traza la línea del recall global. **El título lo escribís vosotros** con vuestro peor subgrupo y sus cifras.

7. **Lista de errores (8 minutos).** Las cancelaciones que el modelo no detectó, de la más segura a la menos:

```
resultados = X_test.copy()
resultados["real"] = real
resultados["pred"] = pred
resultados["prob"] = prob.round(3)
no_detectadas = resultados[(resultados["real"] == 1) & (resultados["pred"] == 0)].sort_values("prob")
print(len(no_detectadas), "cancelaciones no detectadas de", int(real.sum()))
no_detectadas.head(20)
```

   Mirad esas 20 filas una a una y buscad qué tienen en común. Para comprobarlo con números, comparad las no detectadas con **todas** las canceladas (cambiad `canal` y `antelacion_dias` por columnas vuestras):

```
canceladas = resultados[resultados["real"] == 1]
comparacion = pd.DataFrame({"no detectadas": no_detectadas["canal"].value_counts(normalize=True),
                            "todas las canceladas": canceladas["canal"].value_counts(normalize=True)}).round(2)
print(comparacion)
print("Antelación media (días):", round(no_detectadas["antelacion_dias"].mean(), 1), "frente a",
round(canceladas["antelacion_dias"].mean(), 1))
```

   `value_counts(normalize=True)` cuenta qué parte de las filas es de cada canal. En una celda de texto, escribid 2–3 frases («fallamos más con…»). De ahí salen las **limitaciones** de la model card y de la memoria.

8. **Tutoría de sesgos y visualización.** En estos 40 minutos cada equipo tiene su franja: llevad el cuaderno abierto, con la tabla por subgrupos y el gráfico.

Qué deberíais ver. Con el ejemplo de TurisData:

- `Reservas de prueba: 1500` y `AUC 0.744 · recall 0.828 · precisión 0.589 (umbral 0.35).`
- Intervalos aproximados: AUC entre 0.719 y 0.767; recall entre 0.799 y 0.854.
- La tabla por canal: `directo`, 125 cancelaciones y solo 77 detectadas (**recall 0.616, ± 0.085**); `web_intermediaria`, 397 y 361 (0.909). El global (0.828) esconde esa diferencia de 21 puntos.
- En el bucle, el peor subgrupo de `pais_origen` es `sin dato` (recall 0.533 con 15 cancelaciones: el ± sale de 0.252, poca evidencia).
- `118 cancelaciones no detectadas de 688`. Entre ellas pesa el canal `directo` (41 % de las no detectadas frente al 18 % del total de canceladas), y se hicieron con menos antelación (57 días de media frente a 111 en el total de canceladas).

Con vuestros datos, las mismas formas con vuestras cifras: una tabla de métricas con intervalo, 2–4 gráficos con título-conclusión, una tabla por subgrupo y 2–3 frases sobre los errores. Aunque no haya diferencias entre subgrupos, la tabla se entrega.

Hecho cuando: `04_evaluacion.ipynb` tiene la evaluación en test (una vez) con su intervalo, los gráficos, la métrica por subgrupos y la lista de errores, y todo el equipo sabe qué subgrupo va peor.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'prob' is not defined` (o `pred`, `auc`, `X_test`, `UMBRAL`)	No se ejecutó alguna celda de `S13_02`, o Colab se reinició.	Pulsad ▶ por orden desde la primera celda hasta «2. Cálculos».
`NameError: name 'recall_score' is not defined`	Falta la línea `from sklearn.metrics import recall_score, precision_score`.	Ejecutad la celda del apartado 3 entera, empezando por esa línea.
`FileNotFoundError: [Errno 2] No such file or directory: 'modelo_candidato.joblib'`	El fichero del modelo no está en Colab.	Arrastradlo al panel y escribid su nombre exacto.
`KeyError: 'canal'` (con otro nombre)	La columna no se llama así en vuestros datos.	Mirad los nombres con `X_test.columns`.
`TypeError: '<' not supported between instances of 'float' and 'str'`	La columna tiene huecos y Python no sabe ordenarlos.	Usad `.fillna("sin dato")`, como en el ejemplo.
`ZeroDivisionError: division by zero`	Un subgrupo no tiene ninguna cancelación (ningún caso positivo) y no se puede calcular su recall.	Juntadlo con otro subgrupo parecido, o dejadlo fuera y anotad por qué.

Veis…	Qué significa	Qué hacer
Las cifras cambian cada vez que ejecutáis	La partición o el remuestreo no usan semilla fija.	Usad la misma `random_state` que en el Sprint 15 y `np.random.default_rng(SEMILLA)`.

Paso 2 · Abrir la memoria y la model card (día 1, 85 minutos)

1. Repartir las secciones de la memoria entre las personas del equipo.
2. Copiar el esqueleto de la memoria y la plantilla de la model card y rellenar lo que ya sabéis.
3. Dejar el demostrador básico funcionando y anotar lo que falta y quién lo hace.

En sencillo. Un proyecto de IA no termina cuando el modelo funciona: otra persona tiene que poder **entenderlo, probarlo y repetirlo**. Para eso entregáis dos documentos (la **memoria**, de 10–15 páginas, y la **model card**, la ficha de 2 páginas del modelo) y una pequeña aplicación (el **demostrador**) donde alguien prueba el modelo. Hoy no escribís la versión final: montáis el **esqueleto** de cada pieza, repartís quién escribe qué y rellenáis lo que ya sabéis de los sprints 14 y 15. Es como construir una casa: antes de pintar las paredes, planos y reparto de tareas. No hace falta programar: casi todo es copiar plantillas y escribir; en el demostrador solo ejecutáis y adaptáis un cuaderno que ya está hecho.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Memoria técnica	El documento PDF de 10–15 páginas que cuenta el proyecto entero. Portada, índice, referencias y anexos no cuentan.
Esqueleto	La lista de secciones (0 a 9) con lo que debe ir en cada una, antes de escribir.
Requisito R1–R11	Los 11 mínimos del proyecto. El esqueleto trae un mapa que dice en qué sección se demuestra cada uno.
Model card	Ficha de máximo 2 páginas del modelo: para qué sirve, con qué datos, cuánto vale y cuándo no usarlo. Es `docs/model_card.md`.
Demostrador	App o cuaderno donde alguien prueba el modelo sin saber programar. No reentrena: carga un modelo ya entrenado.
`metadatos.json` del demostrador	La ficha del modelo en formato que lee el programa: campos, rangos válidos, AUC y limitaciones. **No es el mismo fichero** que el `metadatos.json` del Sprint 15.
Tablero	El Trello del equipo: *Por hacer → En curso → En revisión → Hecho*. Si no está en el tablero, para el equipo no existe.

Cómo se hace

1. **Repartir las secciones (10 minutos).** Con el esqueleto de la memoria delante, cada sección tiene **una persona responsable** (la escribe con **vuestros** resultados) y otra que la revisa. Un reparto posible para 4 personas (con 3, una coge dos bloques):

Persona	Secciones	De dónde sale el material
A	1 Problema · 2 Datos y EDA	`sprint-14/ficha_proyecto.md`, `plan_datos_licencias.md`, `01_eda.ipynb`
B	3 Metodología · 4 Modelado y experimentos	`sprint-15/registro.csv`, `03_modelos.ipynb`, `justificacion_candidato.md`
C	5 Evaluación · 6 IA responsable	`04_evaluacion.ipynb` (pasos 1 y 3), `riesgos_eticos.md`
D	7 Demostrador · 8 Gestión del proyecto	la carpeta `demostrador/`, el tablero, `backlog_y_sprints.md`

   La sección 9 la escribe el equipo entero y la **0 (resumen ejecutivo) se escribe la última**. Cread en el tablero una tarjeta por sección con su responsable.

2. **Crear la memoria (15 minutos).** En un documento compartido, del programa con el que luego exportaréis el PDF, pegad los títulos de las secciones 0 a 9 del esqueleto. Debajo de cada título, copiad sus viñetas como lista de cosas que hay que contar. Fijaos en el presupuesto de páginas (resumen 1 · problema 1 · datos 2 · metodología 1 · modelado 2 · evaluación 2 · IA responsable 2 · demostrador 1 · gestión 0,5 · conclusiones 1 = 13,5, sobre un máximo de 15). Las secciones 1 a 4 **se cuentan, no se inventan**: salen de los ficheros de la tabla de arriba.

3. **Crear la model card (10 minutos).** Abrid la plantilla en GitHub, pulsad **Raw**, `Ctrl + A` y `Ctrl + C`. En el repositorio del equipo: **Add file → Create new file**, escribid el nombre `sprint-16/docs/model_card.md` (cada barra crea una carpeta), pegad con `Ctrl + V` y pulsad **Commit changes**. Después, en cada apartado del 1 al 5, rellenad así (ejemplo con TurisData):

Apartado	Qué se escribe	Ejemplo
1 · Detalles	Nombre y versión, equipo, fecha y enlace al repositorio, tipo de modelo con la versión de la librería (está en `versiones` del `metadatos.json`), hiperparámetros, semilla, licencia	«Riesgo de cancelación de reservas · TurisData (datos sintéticos) · v1.0». «Regresión logística en scikit-learn 1.9.1». Semilla 42
2 · Uso previsto	Uso principal, quién lo usa, **qué decisión sigue siendo humana** y qué usos no se permiten	«Orientar la gestión de disponibilidad. Una persona decide si pide depósito. No usar para decidir sobre personas concretas»
3 · Datos	La tabla de fuente, tamaño y partición, los sesgos de muestreo y si hay datos personales	«`reservas_turisdata.csv`, 6000 reservas sintéticas; 75 % entrenamiento y 25 % test, estratificado, `random_state`=42. Datos personales: no»
4 · Factores relevantes	Qué grupos o condiciones cambian el rendimiento	«Canal de reserva (en `directo` se detecta menos), reservas sin país y reservas de última hora»
5 · Métricas	Baseline frente al modelo, una métrica técnica y otra de negocio, y la incertidumbre	«AUC: baseline 0,50 → modelo 0,744 (intervalo 95 %: 0,719–0,767). Recall 0,828 con umbral 0,35»

   Lo que aún no sepáis (apartados 6 a 9), dejadlo en blanco: se cierra el día 2. **No escribáis cifras que no estén en vuestro cuaderno.**

4. **Dejar el demostrador funcionando (35 minutos).** Lo hace la persona D con otra de apoyo. Primero **ejecutad el cuaderno `S16_01` tal cual** (hasta la sección «2. La lógica»). Con TurisData crea la carpeta `demostrador/` con `modelo/`, `logica.py`, `app.py` y `api.py`. Luego sustituid los datos y el modelo por los vuestros en la celda de la sección 1 (las líneas con `# >>> EDITA`):

```
reservas = pd.read_csv(dato("reservas_turisdata.csv"))    # >>> EDITA: vuestros datos
NUMERICAS = ["antelacion_dias", "noches", "adultos", "ninos", "precio_noche", "cliente_repetidor",
"cancelaciones_previas"]   # >>> EDITA
CATEGORICAS = ["tipo_habitacion", "canal", "tarifa", "deposito"]    # >>> EDITA
X, y = reservas[NUMERICAS + CATEGORICAS], reservas["cancelada"]     # >>> EDITA: "cancelada" es lo que se predice
```

   Qué hace cada línea:

   - `NUMERICAS` y `CATEGORICAS` son las columnas con las que se entrenó el modelo, divididas en números y en categorías. **Tienen que ser todas las del candidato, con sus nombres exactos**: `logica.py` pide al usuario justo esas.
   - En la celda de abajo (`meta = {…}`) cambiad `nombre`, la regla `min`/`max` y la `etiqueta` de cada campo, y el texto de `limitaciones` (cuándo no fiarse). Si vuestro candidato es un `Pipeline` de scikit-learn ya guardado, copiadlo con `joblib.dump(modelo, "demostrador/modelo/modelo_cancelaciones.joblib")` en vez de reentrenar.
   - Para imágenes, series o un asistente, mirad el apartado «Qué adaptar en vuestro proyecto» del `README_demostrador.md` y preguntad al docente.

   La carpeta de plantilla completa está en `demostrador/`: copiadla a `sprint-16/demostrador/` de vuestro repositorio (en el repositorio del curso, *Code → Download ZIP*) y trabajad en la copia. La carpeta `modelo/` no viene hecha: la crea el cuaderno.

5. **Apuntar lo que falta (15 minutos).** Una tarjeta del tablero por cada pieza pendiente, con responsable y día: validaciones y aviso del demostrador (día 2), pruebas con `pytest` (día 2), apartados 6 a 9 de la model card (día 2), borrador de las secciones 1 a 6 de la memoria (**antes del 9/12**, durante los dos días sin clase), vídeo plan B (día 3). Cada persona escribe en el tablero qué avanza en esos dos días.

Qué deberíais ver.

- **En el cuaderno `S16_01`:** `AUC en test: 0.746` y `Guardado: ['modelo_cancelaciones.joblib', 'metadatos.json']` (el orden de la lista puede variar). La predicción del ejemplo, `{'errores': [], 'probabilidad': 0.812, 'nivel': 'alto', 'tasa_base': 0.459, 'aviso': '…'}`, y la del ejemplo mal escrito, `{'errores': ["'noches' debe estar entre 1 y 30 (has escrito -3).", "'telepatia' no es un valor válido para 'canal'. Opciones: agencia, directo, empresa, web_intermediaria."]}`.
- **En el repositorio:** `sprint-16/docs/model_card.md` con los apartados 1 a 5 rellenos y el 6 al 9 en blanco; en el documento de la memoria, los 10 títulos con una persona responsable; en el tablero, una tarjeta por sección y por pieza pendiente.

Hecho cuando: existe el esqueleto de la memoria con una persona responsable por sección, la model card está empezada y el demostrador básico funciona con vuestro modelo.

Si algo falla

Veis…	Qué significa	Qué hacer
`FileNotFoundError: Falta el modelo. Ejecuta primero el notebook S16_01 (o tu script de entrenamiento) para crear la carpeta 'modelo/'.`	`logica.py` no encuentra `demostrador/modelo/`.	Ejecutad la celda de la sección 1 de `S16_01`, que guarda el modelo y `metadatos.json`.
`NameError: name 'dato' is not defined`	No se ejecutó la primera celda de código («Preparación»).	Pulsad ▶ por orden desde el principio.
`KeyError: 'cancelada'` (con vuestro nombre de columna)	La columna que se predice no se llama así en vuestros datos.	Mirad los nombres con `reservas.columns`.
`ModuleNotFoundError: No module named 'logica'`	La celda que crea `logica.py` no se ha ejecutado.	Ejecutad la celda de la sección 2 antes de usar `logica`.
`ValueError: columns are missing: {'canal'}`	`meta["campos"]` no incluye una columna con la que se entrenó el modelo.	Añadidla a `NUMERICAS` o `CATEGORICAS` y volved a ejecutar las celdas de la sección 1.
El archivo de la model card aparece en la raíz del repositorio	Se creó sin la ruta `sprint-16/docs/`.	Abridlo, pulsad el lápiz y escribid la ruta completa delante del nombre: `sprint-16/docs/model_card.md`.

Paso 3 · Mitigar el sesgo y volver a medir (día 2, 80 minutos)

1. Elegir una medida de mitigación para el sesgo que detectasteis.
2. Aplicarla y volver a medir, por subgrupo, antes y después.
3. Anotar los resultados en el cuaderno y en el registro de experimentos.

En sencillo. Imaginad una báscula que pesa bien casi todo, pero se queda corta con los objetos pequeños. No basta con darse cuenta: hay que ajustarla y **volver a pesar** para comprobar dos cosas: que ahora pesa bien lo pequeño y que no ha empezado a fallar con lo demás. Con el modelo es igual. Una **mitigación** es una medida que toca algo (la decisión, los datos, el entrenamiento) para que al grupo que va peor le vaya mejor. Después se vuelve a medir, grupo a grupo, y se apunta lo que mejoró, lo que empeoró y lo que costó. **No existe una mitigación gratis**, y a veces la que probáis no funciona: eso también es un resultado válido y se anota. Una regla de oro: el número que ajustáis (un umbral, un peso) lo elegís con los datos de **entrenamiento**, nunca mirando el test; el test solo se usa para medir el «después» con la medida ya fijada.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Mitigación	Medida para reducir un sesgo. Hay que medirla, no solo aplicarla.
Antes y después	La misma tabla por subgrupo calculada sin la medida y con ella, para compararlas fila a fila.
Umbral por grupo	Decir «cancelará» a partir de una probabilidad distinta en cada grupo. Un umbral más bajo avisa más y detecta más, pero también da más falsas alarmas.
Reponderar	Dar más peso, al entrenar, a los ejemplos del grupo que va peor.
Validación cruzada (`cross_val_predict`)	Predice cada dato de entrenamiento con un modelo que no lo ha visto. Sirve para decidir **sin tocar el test** (como en `S07_04`).
`.map`	Cambia cada valor de una columna según un diccionario: `{"directo": 0.25}` convierte «directo» en 0,25.
Registro de experimentos	El `registro.csv` del Sprint 15: una fila por prueba.

Cómo se hace

1. **Elegir una medida (5 minutos).** Mirad la tabla por subgrupos del paso 1 y escoged **una** (la que mejor encaje con vuestro sesgo):

Medida	En qué consiste
Reponderar	Dar más peso al entrenar a los ejemplos del subgrupo que va peor
Ajustar el umbral por grupo	Decidir «cancelará» con una probabilidad mínima distinta para cada grupo
Añadir datos	Conseguir más ejemplos del subgrupo que falta
Limitar el uso	Decir en el aviso del demostrador: «no usar el modelo para el grupo X»

   Aquí tenéis código para las dos primeras (con TurisData y su peor subgrupo, el canal `directo`). Para la tercera y la cuarta, mirad el apartado 6.

2. **Guardar el «antes» (5 minutos).** En `04_evaluacion.ipynb`, con las celdas del paso 1 ya ejecutadas (necesitáis la función `medir_subgrupos` y las variables `real`, `pred`):

```
grupo = X_test["canal"].fillna("sin dato").to_numpy()
antes = medir_subgrupos(real, pred, grupo)
avisos_antes = int(pred.sum())
falsas_antes = int(((pred == 1) & (real == 0)).sum())
```

   `antes` es la tabla por canal de siempre y las otras dos variables guardan cuántos avisos y cuántas falsas alarmas hubo. **No cambiéis `pred`**: la medida creará una `pred_nueva` aparte, para poder comparar. Poned vuestra columna donde pone `canal`.

3. **Medida A · umbral por grupo (20 minutos).** Primero, con los datos de entrenamiento, ver qué umbral conviene al grupo:

```
from sklearn.base import clone
from sklearn.model_selection import cross_val_predict, StratifiedKFold

cv = StratifiedKFold(5, shuffle=True, random_state=SEMILLA)
prob_cv = cross_val_predict(clone(modelo), X_ent, y_ent, cv=cv, method="predict_proba")[:, 1]
real_ent = y_ent.to_numpy()
print("Recall global en validación cruzada:", round(recall_score(real_ent, prob_cv >= UMBRAL), 3))
en_directo = (X_ent["canal"] == "directo").to_numpy()
for umbral in [0.35, 0.30, 0.25, 0.20, 0.15]:
    avisa = prob_cv[en_directo] >= umbral
    print(f"umbral {umbral:.2f} → recall {recall_score(real_ent[en_directo], avisa):.3f} · precisión
{precision_score(real_ent[en_directo], avisa):.3f}")
```

   Qué hace cada parte:

   - `clone(modelo)`: una copia **sin entrenar** del modelo, con los mismos ajustes. `cross_val_predict(…, cv=cv, method="predict_proba")[:, 1]` entrena 5 veces con 4/5 de los datos de entrenamiento y predice el 1/5 restante, así que `prob_cv` es una probabilidad «honesta» para cada reserva de entrenamiento.
   - `en_directo`: `True` donde la reserva es del canal `directo`. Cambiad `"directo"` por vuestro peor subgrupo.
   - El bucle prueba cinco umbrales y escribe, para ese grupo, el recall y la precisión. **Elegid el umbral con el que el recall del grupo se acerca al global sin disparar las falsas alarmas.** Esa es una decisión del equipo y se justifica en la memoria.

   Con el umbral elegido, se aplica al test y se mide el «después»:

```
UMBRALES = {"agencia": UMBRAL, "directo": 0.25, "empresa": UMBRAL, "web_intermediaria": UMBRAL}
umbral_fila = X_test["canal"].map(UMBRALES).to_numpy()
pred_nueva = (prob >= umbral_fila).astype(int)
despues = medir_subgrupos(real, pred_nueva, grupo)
```

   `UMBRALES` dice qué umbral usa cada grupo (todos igual que antes, salvo el que queréis mejorar). `.map(UMBRALES)` pone a cada reserva el umbral de su canal, y `pred_nueva` avisa cuando su probabilidad lo alcanza. **Los nombres de las claves deben coincidir letra por letra con los valores de la columna.**

4. **Medida B · reponderar (15 minutos, opcional).** Probad si dar más peso al grupo ayuda; la prueba se hace también con el entrenamiento, no con el test:

```
for peso in [1, 2, 3, 5]:
    pesos = np.where(X_ent["canal"] == "directo", float(peso), 1.0)
    prob_peso = cross_val_predict(clone(modelo), X_ent, y_ent, cv=cv, method="predict_proba", params={"m__sample_weight":
pesos})[:, 1]
    avisa = prob_peso[en_directo] >= UMBRAL
    print(f"peso {peso} → recall en directo {recall_score(real_ent[en_directo], avisa):.3f} · precisión
{precision_score(real_ent[en_directo], avisa):.3f}")
```

   `np.where(condición, a, b)` pone el peso a a las reservas de `directo` y 1 a las demás. `m__sample_weight` es la forma de decirle al modelo (`Pipeline`) «pasa estos pesos a la pieza `m`» (las dos piezas se llaman `prep` y `m`, como en `S13_02`). Si el recall del grupo no sube, la medida **no funciona**: descartadla, anotadlo y probad otra. Comparar dos medidas cuenta como nivel extra.

5. **Medir antes y después y escribir la lectura (20 minutos).**

```
tabla = antes[["subgrupo", "reservas", "recall", "precisión"]].merge(
    despues[["subgrupo", "recall", "precisión"]], on="subgrupo", suffixes=(" antes", " después"))
tabla["cambio en recall (puntos)"] = ((tabla["recall después"] - tabla["recall antes"]) * 100).round(1)
print(tabla.to_string(index=False))
print(f"Recall global: {recall_score(real, pred):.1%} → {recall_score(real, pred_nueva):.1%}")
print(f"Avisos: {avisos_antes} → {int(pred_nueva.sum())} · falsas alarmas: {falsas_antes} → {int(((pred_nueva == 1) &
(real == 0)).sum())}")
```

   `.merge(…, on="subgrupo")` junta las dos tablas fila a fila por el nombre del subgrupo y `suffixes` añade «antes» y «después» a las columnas repetidas. Debajo, **una frase de lectura honesta** con vuestras cifras: «El recall de [grupo] sube del [X] % al [Y] % y el de [resto] [cambia/no cambia]; a cambio hay [N] falsas alarmas más». Copiad la tabla a una celda de texto.

6. **Si el sesgo no se puede arreglar (en el mismo tiempo).** Para **añadir datos**, decid cuántos casos del grupo faltan y cómo se conseguirían; si no hay forma, es una limitación. Para **limitar el uso**, escribidlo en el campo `limitaciones` del demostrador (paso 4) y en la model card. Esconderlo no es una opción.

7. **Registro de experimentos (15 minutos).** Una fila nueva en `registro.csv` con la medida, el antes y después y la decisión. Si copiasteis `registrar` a `utilidades.py` en el Sprint 15 (S15_01), usadla; si no, añadid la fila a mano con las mismas 12 columnas:

```
registrar("Ana", "RegresionLogistica+umbral_directo", UMBRALES, DATOS_HASH, SEMILLA, "recall_directo", 0.800, 0, 0,
          "Mitigación del canal directo. Test: recall 61,6 % → 79,2 %; +52 falsas alarmas; resto de canales igual")
```

   Los argumentos van en este orden: quién la hace (vuestro nombre), modelo, ajustes (`UMBRALES`), huella de los datos (`DATOS_HASH`, de `03_modelos.ipynb`), semilla, nombre de la métrica, su valor en validación cruzada (0,800 es el recall de `directo` con umbral 0,25 de la tabla del apartado 3), su desviación (0 si no la calculasteis: decidlo en `notas`), el tiempo y las notas.

Qué deberíais ver. Con el ejemplo de TurisData:

- `Recall global en validación cruzada: 0.834` y, para `directo`, recall 0.597 con umbral 0.35 que sube a 0.800 con 0.25 (precisión de 0.521 a 0.464).
- Con el peso (opcional): recall de `directo` en 0.597, 0.600, 0.600 y 0.591 para los pesos 1, 2, 3 y 5. **No mejora**: reponderar aquí no sirve.
- La tabla de antes y después: `directo` pasa de recall 0.616 a 0.792 (**+17.6 puntos**, precisión de 0.500 a 0.434) y los otros tres canales no cambian (0.0).
- `Recall global: 82.8% → 86.0%` y `Avisos: 967 → 1041 · falsas alarmas: 397 → 449`: la mejora cuesta 52 falsas alarmas más. Con los costes de `S13_02` (50 € por cancelación no detectada, 30 € por falsa alarma) la medida sale unos 460 € más cara en las 1500 reservas de prueba: la equidad tiene precio.

Hecho cuando: hay una mitigación aplicada, medida por subgrupo antes y después (con su coste) y anotada en el cuaderno y en el registro de experimentos.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'medir_subgrupos' is not defined` (o `real`, `pred`, `prob`)	Colab se reinició o no se ejecutaron las celdas del paso 1.	Pulsad ▶ por orden desde la primera celda del cuaderno.
`ValueError: Pipeline.fit does not accept the sample_weight parameter`	Habéis escrito `sample_weight` en vez de `m__sample_weight`.	Usad el nombre de la pieza del `Pipeline` con dos guiones bajos.
El recall de un grupo cae a 0.000 después de aplicar los umbrales	Una clave de `UMBRALES` no coincide con el valor de la columna (por ejemplo `"Directo"` en vez de `"directo"`): `.map` deja un hueco (NaN) y un hueco nunca alcanza el umbral, así que esa reserva no avisa nunca.	Comprobad `X_test["canal"].map(UMBRALES).isna().sum()`: debe dar 0.
`TypeError: cross_val_predict() got an unexpected keyword argument 'params'`	Una versión de scikit-learn anterior a la 1.4, donde el argumento se llama `fit_params`.	Cambiad `params=` por `fit_params=` o preguntad al docente.
El recall del grupo casi no se mueve (0.597 → 0.600)	La medida no funciona con este modelo.	Anotadlo como resultado, descartadla y probad otra.
Sube el recall del grupo pero bajan la precisión y el número de falsas alarmas se dispara	Es el precio de la medida, no un fallo.	Cuantificadlo (avisos y falsas alarmas) y explicadlo en la lectura.
Las cifras del «antes» no coinciden con las del paso 1	Habéis cambiado `UMBRAL`, la partición o el modelo entre medias.	Volved a ejecutar el cuaderno desde la primera celda sin tocar esos tres.

Paso 4 · Pulir el demostrador y probarlo en limpio (día 2, 40 minutos)

1. Adaptar el aviso de incertidumbre y las validaciones de `logica.py` a vuestro proyecto.
2. Probar a mano cinco entradas: tres normales, una imposible y una vacía.
3. Escribir tres pruebas propias con `pytest` y fijar las versiones en `requirements.txt`.
4. Lanzarlo en una carpeta nueva siguiendo solo el README.

En sencillo. Hasta ahora el demostrador funciona con los datos que vosotros le dais. Hoy lo preparáis para alguien que no sabe nada del modelo: que no se rompa cuando escribe una tontería, que le avise de que el modelo se equivoca y que arranque en un ordenador que no es el vuestro. Es como probar una receta en la cocina de un amigo: si solo sale bien en la vuestra, no está terminada. Las «pruebas automáticas» son comprobaciones de sentido común que el ordenador repite en un segundo (por ejemplo, que una reserva flexible y sin depósito tiene más riesgo que una no reembolsable). No hace falta programar a fondo: cambiáis unos textos, pegáis unos bloques que ya funcionan y escribís unas pocas órdenes que os damos hechas.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Validación	Comprobar que lo que escribe quien usa el demostrador tiene sentido: un número donde toca un número, dentro del rango permitido.
Regla entre campos	Validación que mira dos campos a la vez (por ejemplo, noches × precio no puede pasar de 5000 €). La validación campo a campo no la detecta.
Prueba automática	Función que se llama `test_…` y comprueba algo con `assert` («esto tiene que ser cierto»). Si no lo es, la prueba falla.
`pytest`	Programa que encuentra todas las pruebas y las ejecuta. Dibuja un símbolo por prueba: `.` pasa, `F` falla, `s` se salta.
`requirements.txt`	Lista de las librerías que necesita el proyecto, cada una con su versión fija (`scikit-learn==1.9.1`).
Versión fija	El número exacto de una librería, con `==`. Sin él, otra persona instala una versión más nueva y el modelo puede no cargar o dar otros números.
Entorno virtual (`.venv`)	Carpeta donde se instalan las librerías de un solo proyecto. Sirve para probar en un «ordenador limpio».
Terminal	Ventana donde se escriben órdenes como `pytest -q`. En Visual Studio Code: menú **Terminal → Nuevo terminal**.

Cómo se hace

1. **Preparar la carpeta y la terminal (2 minutos).** Trabajad con una copia de `demostrador/` que tenga dentro la carpeta `modelo/` con `modelo_cancelaciones.joblib` y `metadatos.json` (la crea el cuaderno `S16_01` o vuestro entrenamiento). Este paso se hace en un ordenador, no en Colab: Streamlit no se abre en Colab. Si el modelo está en Colab, comprimid la carpeta con `!zip -r demostrador.zip demostrador` y descargad el ZIP desde el panel de archivos. En Visual Studio Code: **Archivo → Abrir carpeta** (elegid `demostrador`) y **Terminal → Nuevo terminal**. Escribid `ls` (Mac y Linux) o `dir` (Windows): tienen que salir `app.py`, `logica.py`, `modelo` y `tests`.
2. **Escribir el aviso de incertidumbre (4 minutos).** Abrid `modelo/metadatos.json`. Cada resultado del demostrador lleva un aviso que construye `logica.py` con dos campos de ese archivo: `auc_test` (no lo toquéis: es la cifra del modelo) y `limitaciones` (vuestro texto, el último campo). Reescribid `limitaciones` con tres cosas: con qué datos se entrenó, dónde falla (lo que ya sabéis por los pasos 1 y 3 y por la model card, apartado 4) y qué usos no se permiten. Ejemplo con TurisData:

   ```
     "limitaciones": "Entrenado con datos sintéticos de un solo alojamiento ficticio. Detecta peor las cancelaciones del canal directo (ver model card, apartado 4). No usar para decidir sobre personas concretas: solo orienta la gestión de disponibilidad."
   ```

   Qué hay que respetar:

   - Dejad las comillas, escribid todo en **una sola línea** y no pongáis coma al final (es el último campo del archivo).
   - El `auc_test` tiene que coincidir con el AUC del paso 1 y de la model card. En la plantilla de TurisData no coinciden (0,746 en el demostrador y 0,744 en el cuaderno de evaluación) porque el demostrador usa 11 variables y otra partición. En vuestro proyecto el modelo es el mismo: las cifras tienen que ser iguales.
   - Si volvéis a ejecutar la celda del cuaderno que guarda el modelo, este archivo se sobrescribe.
   - Si vuestro demostrador es un asistente, tiene que decir que es una IA y citar sus fuentes; si es de imágenes, avisar de que una imagen rara queda fuera de lo que conoce. Está en «Qué adaptar en vuestro proyecto» del `README_demostrador.md`.

3. **Añadir una regla que cruce dos campos (7 minutos).** Abrid `logica.py`. Primero pegad esta función justo antes de `def nivel_riesgo(probabilidad):`:

```
def validar_coherencia(reserva):
    """Reglas que cruzan varios campos. Solo se llama cuando cada campo ya es válido por separado."""
    errores = []
    importe = reserva["noches"] * reserva["precio_noche"]
    if importe > 5000:
        errores.append(f"El importe total supera los 5000 € (son {importe:.0f} €). Revisa las noches y el precio.")
    return errores
```

Qué hace cada línea:

- `def validar_coherencia(reserva):` crea la función. `reserva` es el diccionario con lo que ha escrito la persona.
- `errores = []` es la lista donde se apuntan los problemas. Vacía significa «todo bien».
- `importe = reserva["noches"] * reserva["precio_noche"]` multiplica dos campos. Los nombres entre comillas tienen que ser **exactamente** los de `meta["campos"]`.
- `if importe > 5000:` se cumple solo si la regla se rompe. `errores.append(f"…")` añade el mensaje; `{importe:.0f}` escribe el número sin decimales. El mensaje dice qué falla y qué hacer.
- `return errores` devuelve la lista.

Después, dentro de `predecir_reserva`, sustituid estas dos líneas:

```
    errores = validar_reserva(reserva, meta)
    if errores:
        return {"errores": errores}
```

por estas cuatro:

```
    errores = validar_reserva(reserva, meta)
    if not errores:
        errores = validar_coherencia(reserva)
    if errores:
        return {"errores": errores}
```

Qué hace el cambio: `if not errores:` significa «si no hay errores». La regla nueva solo se mira cuando cada campo ya es válido; si se mirase antes, un campo que falta daría `KeyError: 'noches'`. Cambiad la regla de ejemplo por una de **vuestro** proyecto (por ejemplo, una fecha de salida anterior a la de entrada). La regla «niños sin adultos» del ejercicio 1 de `S16_01` aquí no hace falta: `adultos` ya tiene mínimo 1.

4. **Probar a mano cinco entradas (6 minutos).** Cread en `demostrador/` el archivo `probar_a_mano.py` (en VS Code: botón derecho en la carpeta → **Nuevo archivo**) con este código:

```
from logica import cargar_modelo, predecir_reserva

modelo, meta = cargar_modelo()

normal = {"antelacion_dias": 120, "noches": 5, "adultos": 2, "ninos": 1, "precio_noche": 95.0, "cliente_repetidor": 0,
          "cancelaciones_previas": 0, "tipo_habitacion": "estandar", "canal": "web_intermediaria",
          "tarifa": "flexible", "deposito": "sin_deposito"}

pruebas = {
    "normal 1": normal,
    "normal 2": {**normal, "antelacion_dias": 7, "tarifa": "no_reembolsable", "deposito": "total", "canal": "directo"},
    "normal 3": {**normal, "adultos": 1, "ninos": 0, "noches": 3, "tipo_habitacion": "suite", "precio_noche": 180.0,
"cliente_repetidor": 1},
    "imposible": {**normal, "antelacion_dias": 5000},
    "vacía": {},
}

for nombre, entrada in pruebas.items():
    r = predecir_reserva(entrada, modelo, meta)
    if r["errores"]:
        print(f"{nombre}: {len(r['errores'])} errores · {r['errores'][0]}")
    else:
        print(f"{nombre}: riesgo {r['nivel']} ({r['probabilidad']:.0%}) · media histórica {r['tasa_base']:.0%}")
```

Qué hace cada línea:

- `from logica import …` trae las dos funciones de `logica.py`; por eso el archivo tiene que estar en la misma carpeta.
- `modelo, meta = cargar_modelo()` carga el modelo y `metadatos.json`.
- `normal = {…}` es una reserva completa y válida. Cambiad los nombres y valores por los campos de **vuestro** proyecto, escritos igual que en `meta["campos"]`.
- `{**normal, "antelacion_dias": 7}` copia `normal` y cambia solo lo que indicáis. Así cada prueba son dos o tres datos, no once.
- `"imposible"` tiene que ser un valor absurdo de vuestro proyecto (una edad de 250, una fecha del año 3000). `"vacía": {}` es una entrada sin ningún dato.
- El bucle `for` ejecuta las cinco y escribe una línea por cada una. `{r['probabilidad']:.0%}` muestra el número como porcentaje sin decimales.

Ejecutadlo en la terminal con `python probar_a_mano.py`. Las tres normales dan un nivel y un porcentaje; la imposible y la vacía dan un mensaje claro y **ningún error rojo de Python**. Si sale un error rojo, el demostrador no está listo: arreglad `logica.py` hasta que las cinco funcionen.

5. **Escribir tres pruebas propias (7 minutos).** Abrid `tests/test_logica.py`. Cambiad el diccionario RESERVA (la línea que termina en `# >>> ADAPTA`) por una reserva de vuestro proyecto. Después pegad **al final del archivo** estas tres pruebas:

```
def test_flexible_sin_deposito_tiene_mas_riesgo(modelo_y_meta):
    modelo, meta = modelo_y_meta
    flexible = {**RESERVA, "tarifa": "flexible", "deposito": "sin_deposito"}
    segura = {**RESERVA, "tarifa": "no_reembolsable", "deposito": "total"}
    assert predecir_reserva(flexible, modelo, meta)["probabilidad"] > predecir_reserva(segura, modelo, meta)
["probabilidad"]

def test_entrada_vacia_avisa_de_todos_los_campos(modelo_y_meta):
    modelo, meta = modelo_y_meta
    r = predecir_reserva({}, modelo, meta)
    assert len(r["errores"]) == len(meta["campos"]) and "probabilidad" not in r

def test_importe_demasiado_alto_se_rechaza(modelo_y_meta):
    modelo, meta = modelo_y_meta
    r = predecir_reserva({**RESERVA, "noches": 20, "precio_noche": 400.0}, modelo, meta)
    assert r["errores"] and r["errores"][0].startswith("El importe total supera los 5000 €")
```

   Qué hace cada línea:

   - `def test_…(modelo_y_meta):` el nombre **tiene que empezar por `test_`** o `pytest` no la ejecuta. `modelo_y_meta` es la carga del modelo que ya trae el archivo: no la toquéis.
   - `{**RESERVA, "tarifa": …}` copia la reserva de ejemplo y cambia solo esos campos.
   - `assert A > B` significa «A tiene que ser mayor que B». Si no lo es, la prueba falla y `pytest` lo cuenta.
   - La primera prueba es de **sentido común del modelo**, la segunda de **entrada vacía** y la tercera de **entrada imposible** (usa la regla del punto 3). Sustituidlas por tres reglas de vuestro proyecto, una de cada tipo.

   En la terminal, desde `demostrador/`, escribid `pytest -q`. Si `pytest` no está instalado, `pip install pytest` antes.

6. **Fijar las versiones (4 minutos).** Abrid `requirements.txt`. La plantilla trae las versiones del entorno del curso, que **no son necesariamente las vuestras**. Dejad solo lo que usáis, cada librería con == y con la versión **con la que guardasteis el modelo**: están en `modelo/metadatos.json`, campo `versiones` (`python`, `scikit-learn`, `pandas`, `numpy`). Para `joblib`, `streamlit` y `pytest` escribid `pip show joblib streamlit pytest` en la terminal y copiad las líneas `Version:`. Quitad `ipywidgets`, `fastapi` y `uvicorn` si no usáis los *widgets* ni `api.py`. Con TurisData queda:

```
numpy==2.5.3
pandas==2.3.3
scikit-learn==1.9.1
joblib==1.6.0
streamlit==1.64.0
pytest==9.1.1
```

   Si la versión de `scikit-learn` no coincide con la del modelo, al cargarlo sale un `InconsistentVersionWarning` (mirad la tabla de abajo). Después añadid arriba de `README_demostrador.md` cuatro líneas con la versión de Python (`python` de `versiones`) y las órdenes exactas:

```
Python: 3.13 (la del campo "versiones" de modelo/metadatos.json)
Instalar: pip install -r requirements.txt
Lanzar: streamlit run app.py   (se abre en http://localhost:8501)
Probar: pytest -q
```

7. **Probarlo en limpio con otra persona (8 minutos).** Lo hace alguien que **no ha escrito** el código, siguiendo **solo el README**, con la persona que lo escribió al lado y sin ayudar. Copiad `demostrador/` a una carpeta nueva (por ejemplo, en el Escritorio), abridla en VS Code, abrid una terminal y escribid:

   ```
   python -m venv .venv
   .venv\Scripts\activate
   source .venv/bin/activate
   pip install -r requirements.txt
   pytest -q
   streamlit run app.py
   ```

   Qué hace cada orden:

   - `python -m venv .venv` crea el entorno limpio en la carpeta `.venv`.
   - Las dos siguientes **activan** el entorno: la de `.venv\Scripts\activate` es para Windows y la de `source …` para Mac y Linux. Escribid solo la que os toque. Si ha funcionado, la línea de la terminal empieza por `(.venv)`.
   - `pip install -r requirements.txt` instala lo de la lista. Tarda unos minutos.
   - `pytest -q` pasa las pruebas y `streamlit run app.py` abre la app en el navegador.

   En la app probad las mismas entradas: una normal, una imposible (el cuadro amarillo tiene que decir qué campo falla) y una vacía (borrad el número de una casilla). Cada vez que quien prueba tenga que **adivinar** algo, apuntadlo y añadidlo al README.

8. **Subir los cambios (2 minutos).** Subid a `sprint-16/demostrador/` del repositorio los archivos cambiados (`logica.py`, `tests/test_logica.py`, `requirements.txt`, `README_demostrador.md`, `probar_a_mano.py` y la carpeta `modelo/`) con **Add file → Upload files**, como explica Cómo entregar en GitHub. **No subáis** la carpeta `.venv/`, que pesa cientos de megas, ni `__pycache__/`.

Qué deberíais ver.

- **`python probar_a_mano.py`**, con el ejemplo de TurisData:

  ```
  normal 1: riesgo alto (81%) · media histórica 46%
  normal 2: riesgo bajo (4%) · media histórica 46%
  normal 3: riesgo alto (63%) · media histórica 46%
  imposible: 1 errores · 'antelacion_dias' debe estar entre 1 y 400 (has escrito 5000).
  vacía: 11 errores · Falta el campo 'antelacion_dias'.
  ```

- **`pytest -q`:** `........` y debajo `8 passed in 1.4s` (el tiempo cambia). Son las 5 pruebas de la plantilla y vuestras 3.
- **`streamlit run app.py`:** en la terminal, `You can now view your Streamlit app in your browser.` y `Local URL: http://localhost:8501`. En el navegador, el formulario. Con los valores que trae por defecto, **Calcular riesgo** da `Probabilidad de cancelación 25%` (`-20% frente a la media`), `Nivel de riesgo BAJO`, una barra y un cuadro azul con el aviso. Con una antelación de 5000 sale un cuadro amarillo: `No se puede calcular todavía:` y `- 'antelacion_dias' debe estar entre 1 y 400 (has escrito 5000).` Si borráis un número, `- Falta el campo …`. Nunca un cuadro rojo con un error de Python.

Hecho cuando: el demostrador se lanza en una carpeta limpia con las órdenes del README, no se rompe con las cinco entradas de prueba, el aviso de incertidumbre sale en cada resultado y `pytest -q` pasa con vuestras tres pruebas.

Si algo falla

Veis…	Qué significa	Qué hacer
`....F...` y al final `1 failed, 7 passed`	Una prueba ha fallado: hay un error en el código o el modelo contradice el sentido común.	Leed las líneas que empiezan por `>` y `E`: dicen qué comparación falló. Si es el modelo quien contradice la regla, no cambiéis la prueba para que pase: apuntadlo como límite en la model card.
`ssss.sss` y `1 passed, 7 skipped`, o en la app un cuadro rojo con `Falta el modelo. Ejecuta primero el notebook S16_01…`	No encuentra la carpeta `modelo/`, así que se saltan las pruebas.	Comprobad que `modelo/` está al lado de `logica.py` con sus dos archivos y que lanzáis `pytest -q` desde `demostrador/`.
`ModuleNotFoundError: No module named 'streamlit'` (o `'sklearn'`, `'pytest'`)	La librería no está instalada en el entorno que estáis usando.	Con el entorno activado (la línea empieza por `(.venv)`), `pip install -r requirements.txt`.

Veis…	Qué significa	Qué hacer
`python: can't open file '… probar_a_mano.py': [Errno 2] No such file or directory`	La terminal no está en la carpeta `demostrador/`.	Abrid la carpeta en VS Code (**Archivo → Abrir carpeta**) y abrid una terminal nueva.
`KeyError: 'noches'` (o el nombre de otro campo)	`validar_coherencia` se ejecutó con un campo que falta.	Llamadla solo dentro de `if not errores:`, como en el punto 3.
`InconsistentVersionWarning: Trying to unpickle estimator Pipeline from version 1.9.1 when using version 1.8.0`	`requirements.txt` fija otra versión de `scikit-learn` que la del modelo guardado.	Poned en `requirements.txt` la versión de `versiones` en `metadatos.json` y repetid `pip install -r requirements.txt`.
`"python" no se reconoce como un comando interno o externo` (Windows) o `command not found: python` (Mac)	La terminal no encuentra Python.	Probad `python3` en lugar de `python`. Si tampoco, usad la terminal de Visual Studio Code del aula.

Paso 5 · Cerrar la model card y completar la memoria (día 2, 80 minutos)

1. Clasificar el sistema según el RGPD y el AI Act y cerrar el registro de riesgos.
2. Terminar la model card con las cifras finales (los apartados 6 a 9).
3. Completar el borrador de la memoria con todos los apartados.

En sencillo. Hasta hoy habéis construido el modelo; ahora hay que **contárselo a otra persona con pruebas**. Es como entregar un coche: no basta con que ande, hay que dar la ficha técnica (la model card), la lista de lo que puede fallar y qué se ha hecho para evitarlo (el registro de riesgos) y el manual completo (la memoria). La regla de oro es que **cada afirmación se pueda comprobar**: «hemos cuidado la privacidad» no vale; «no hay datos personales, ver `plan_datos_licencias.md`, apartado 6» sí. No hace falta saber de leyes ni de Python: casi todo es rellenar los documentos que ya tenéis con las cifras de vuestros cuadernos. Una pequeña parte de código os saca esas cifras ya escritas, para que no las copiéis a mano y no se cuele un error.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Model card	La ficha de máximo 2 páginas del modelo. Hoy se rellenan los apartados 6 a 9 (subgrupos, riesgos, limitaciones, mantenimiento).
Subgrupo	Una parte de los datos que tiene sentido separar (un canal de venta, un país). El apartado 6 dice cómo rinde el modelo en cada uno.
RGPD	La ley europea de protección de datos personales. Pregunta: ¿hay datos de personas?
AI Act	El reglamento europeo de IA. Clasifica los sistemas por riesgo: inaceptable, alto, limitado o mínimo.
Evidencia	Algo que se puede comprobar en el repositorio: una ruta de archivo y el apartado donde se ve.
Riesgo residual	El riesgo que queda después de mitigar y que decidís aceptar y contar al cliente.
Supervisión humana	Que una persona revisa y decide; el modelo solo orienta.
Pie de figura	La frase bajo un gráfico: «Figura 3. Lo que demuestra. Fuente». Es lo que lee quien hojea la memoria.

Cómo se hace

1. **Repartir y abrir los archivos (3 minutos).** Quien rellena cada cosa:

Persona	Hoy
C	Clasificación y registro de riesgos (puntos 2 y 3), apartados 6 y 7 de la model card, sección 5 y 6 de la memoria
D	Apartados 8 y 9 de la model card (el 8 tiene que decir lo mismo que `limitaciones` del demostrador, paso 4), secciones 7 y 8 de la memoria
B	Apartado 5 de la model card con las cifras finales, secciones 3 y 4 de la memoria
A	Revisar los apartados 1 a 4 de la model card, secciones 1 y 2 de la memoria

   Abrid `sprint-16/docs/model_card.md` (el del paso 2) y el documento de la memoria. El registro de riesgos lo copiáis ahora: abrid `sprint-14/riesgos_eticos.md`, pulsad **Raw**, `Ctrl + A` y `Ctrl + C`; luego **Add file → Create new file**, nombre `sprint-16/docs/riesgos_eticos.md`, `Ctrl + V` y **Commit changes**. Para editar un archivo, el lápiz ; **cada fila de una tabla va en una sola línea**.

2. **Clasificar el sistema (8 minutos).** En `riesgos_eticos.md`, **Parte B**. Una respuesta por fila, con su justificación y, si procede, la ruta de la evidencia. La plantilla orienta así: un sistema que solo predice demanda agregada suele ser de riesgo **mínimo**; un chatbot que informa a personas tiene obligaciones de **transparencia** (riesgo **limitado**); uno que evalúe o decida sobre el acceso de personas a educación, empleo o servicios esenciales es de riesgo **alto**. **Llevad vuestra respuesta escrita al docente antes de darla por buena**, como pide la plantilla. Ejemplo con TurisData:

Cuestión	Respuesta de ejemplo
¿Datos personales?	No. Datos sintéticos de una empresa ficticia; ninguna columna lleva nombres (`plan_datos_licencias.md`, apartado 6).
Base legal / finalidad	No aplica por no haber datos personales. Finalidad: orientar la gestión de disponibilidad.
¿Evaluación de impacto (EIPD)?	No: no hay datos personales ni decisiones sobre personas.
Nivel de riesgo AI Act	**Mínimo**: apoya la gestión de disponibilidad y no decide sobre el acceso de personas a educación, empleo ni servicios esenciales.
Obligaciones que se derivan	Avisar de que es una estimación y de sus límites (demostrador), que una persona decida si se pide depósito y documentarlo (model card).
¿Decide sobre personas?	No. La model card prohíbe usarlo para decidir sobre personas concretas.

3. **Cerrar el registro de riesgos (12 minutos).** Tres cosas, en este orden:
   - **Parte A, columna S16.** Para cada pregunta, un símbolo (*** *cumplido con evidencia*,** ***◐** parcial, *** *no cumplido*, *n/a** no aplica, y por qué) y, en «Evidencia», la ruta y el apartado. Un sin evidencia es un ◐. Ejemplo: «¿Se ha medido el rendimiento por subgrupos?» → `sprint-16/notebooks/04_evaluacion.ipynb` y model card, apartado 6.
   - **Parte C, riesgos altos.** Nivel = P × I (cada uno de 1 a 3): 1-2 bajo, 3-4 medio, **6-9 alto**. Cada riesgo alto necesita estado y evidencia. Así se rellena una fila (R-01 de TurisData):

Campo	Qué se escribe	Ejemplo
Riesgo	Qué puede ir mal y a quién afecta, con cifra si la hay	En el canal `directo` se detecta el 61,6 % de las cancelaciones frente al 82,8 % global: el alojamiento pierde ingresos en ese canal
Tipo · P · I · Nivel	Datos, modelo, uso, legal o técnico; probabilidad; impacto; producto	modelo · 3 · 2 · 6 (alto)
Medida de mitigación	Qué se hizo	Umbral propio de 0,25 para el canal `directo`
Responsable	Una persona	Ana
Estado	`abierto`, `mitigado` o `aceptado` (el sprint a veces dice «pendiente»: es `abierto`)	mitigado
Cómo se evaluó	Prueba, métrica y resultado, con la ruta	Test: recall de `directo` 61,6 % → 79,2 %; +52 falsas alarmas (`04_evaluacion.ipynb`, sección de mitigación)

   - **Parte D, cierre.** Cuatro respuestas: riesgos altos mitigados y su evidencia, **riesgos residuales** que aceptáis y comunicáis, limitaciones y usos no permitidos (pasan a la model card), fecha y firmas. Si un riesgo alto sigue `abierto`, o lo mitigáis hoy o lo aceptáis y lo contáis aquí.
4. **Apartado 6 de la model card: subgrupos (6 minutos).** Las cifras no se escriben a mano. En el cuaderno `04_evaluacion.ipynb`, después de la mitigación del paso 3, ejecutad:

```
for estado, tabla_estado, prediccion in [("antes", antes, pred), ("después", despues, pred_nueva)]:
    global_ = recall_score(real, prediccion)
    for _, f in tabla_estado.iterrows():
        dif = (f["recall"] - global_) * 100
        print(f"| canal = {f['subgrupo']} ({estado}) | {f['reservas']} | recall {f['recall']:.1%} (± {f['±'] * 100:.1f})
| {dif:+.1f} puntos | |")
```

   Qué hace cada línea:
   - `antes` y `despues` son las tablas del paso 3; `pred` y `pred_nueva`, los avisos sin y con la medida. Si dan `NameError`, volved a ejecutar el cuaderno desde la primera celda.
   - `recall_score(real, prediccion)` es el recall global de cada estado y `dif` lo que se aleja de él cada subgrupo, en puntos.
   - El `print(f"| … |")` escribe cada subgrupo como **una fila ya en formato de tabla**. Copiad las filas que os interesan debajo de la cabecera del apartado 6. La última columna, «¿Es aceptable?», la escribís vosotros.

   Con TurisData salen filas como `| canal = directo (antes) | 388 | recall 61.6% (± 8.5) | -21.2 puntos | |`. Dejadlo así en el apartado 6 (cambiad el punto por la coma si queréis):

Subgrupo	n	Métrica	Diferencia con el global	¿Es aceptable?
canal = directo (antes)	388	recall 61,6 % (± 8,5)	−21,2 puntos	No: se escapan 4 de cada 10 cancelaciones
canal = directo (después)	388	recall 79,2 % (± 7,1)	−6,8 puntos	Sí, con coste: 52 falsas alarmas más
canal = empresa	79	recall 73,7 % (± 14,0)	−12,3 puntos	Cifra poco fiable: solo 38 cancelaciones
canal = web_intermediaria	738	recall 90,9 % (± 2,8)	+4,9 puntos	Sí

   El «antes» y el «después» se comparan **cada uno con el global de su estado** (82,8 % y 86,0 %). Añadid una fila por cada subgrupo con diferencia grande y casos suficientes; los que tienen pocos casos, marcadlos como «poco fiable».

5. **Apartados 5, 7, 8 y 9 de la model card (6 minutos).**

Apartado	Qué se escribe	Ejemplo con TurisData
5 · Métricas	Las cifras finales del paso 1, con intervalo	«AUC: baseline 0,50 → modelo 0,744 (95 %: 0,719–0,767). Recall 0,828 (0,799–0,854), precisión 0,589, umbral 0,35. Coste en 1500 reservas de prueba: 34.400 € sin modelo → 17.810 € con él»
7 · Ética y riesgos	La clasificación de la Parte B, los 2 o 3 riesgos principales con cómo se evaluó la mitigación, supervisión humana y transparencia	«Riesgo mínimo; sin datos personales. R-01, sesgo por canal: umbral propio para `directo`, recall 61,6 % → 79,2 %. Supervisión: una persona decide si pide depósito. Transparencia: cada resultado lleva el aviso con el AUC y las limitaciones»
8 · Limitaciones	Cuándo **no** fiarse (casos de vuestros análisis), recomendaciones y qué falta antes de un despliegue real. **Mismo texto que `limitaciones` del demostrador**	«No fiarse con el canal `directo`, con reservas sin país (15 cancelaciones en el test: cifra poco fiable) ni con otro alojamiento distinto del ficticio. Antes de un despliegue real: datos reales y vigilar el recall por canal»
9 · Contacto	Una persona responsable y cada cuánto se reevalúa	«Ana · se reevalúa cuando haya reservas nuevas y siempre que cambie la política de cancelaciones»

Si adoptáis la medida del paso 3, decid en el apartado 7 que el sistema final **usa** el umbral propio. Comprobad que no queda ningún _____ ni `[Nombre del modelo]` (buscadlos con `Ctrl + F`) y que el archivo cabe en 2 páginas: tablas y frases cortas.

6. **Memoria: todas las secciones en borrador (40 minutos).** «Borrador» significa que **cada sección tiene texto**, aunque sea áspero. Cada persona escribe **sus** secciones (las del paso 2) en el documento compartido, con **cifras copiadas de los cuadernos** (no de memoria) y con lo que dice el esqueleto en cada una. Lo que falte, escribidlo como `[PENDIENTE: qué falta]`, para encontrarlo luego con `Ctrl + F`.

Sección (páginas)	Qué se cuenta	De dónde sale
1 Problema (1)	Contexto, **decisión** que apoya y quién la toma, métrica de negocio y técnica con umbral de éxito, valor esperado con supuestos	`ficha_proyecto.md`
2 Datos (2)	Tabla de fuentes con licencia, defectos de calidad y su tratamiento, sesgos de muestreo, 2-3 figuras del EDA	`plan_datos_licencias.md`, `01_eda.ipynb`
3 Metodología (1)	Partición anti-fuga, baseline, métricas, validación cruzada, semillas, enlace a `registro.csv`	`plan_datos_licencias.md`, `03_modelos.ipynb`
4 Modelado (2)	Baseline y al menos 3 modelos, tabla con media ± desviación y tiempo, por qué se eligió el candidato	`registro.csv`, `justificacion_candidato.md`
5 Evaluación (2)	Métricas en test con intervalo, gráficos, análisis de errores con ejemplos, generalización	`04_evaluacion.ipynb` (paso 1)
6 IA responsable (2)	Resumen del checklist, tabla de riesgos altos, subgrupos y mitigación medida, clasificación, limitaciones	`riesgos_eticos.md`, model card, paso 3
7 Demostrador (1)	Qué hace, captura, errores del usuario, cómo reproducirlo, plan B	`demostrador/`, paso 4
8 Gestión (0,5)	Roles y rotación, backlog y sprints, mejoras entre retros (acción → efecto), reparto de trabajo	tablero, `backlog_y_sprints.md`
9 Conclusiones (1)	Qué se recomienda al cliente, qué **no** hacer, límites y trabajo futuro	todo lo anterior

La sección 0 (resumen) se escribe el día 3. Ejemplo de párrafo de la sección 6 con las cifras del paso 3:

```
Medimos el recall (cancelaciones detectadas) por canal en las 1500 reservas de prueba.
El global es del 82,8 %, pero en el canal directo es del 61,6 % (± 8,5 puntos; 77 de 125
cancelaciones). Aplicamos un umbral propio para ese canal (0,25 en lugar de 0,35), elegido
con validación cruzada sobre el entrenamiento. Tras la medida el recall de directo sube al
79,2 % y el global al 86,0 %; los otros canales no cambian. El precio son 52 falsas alarmas
más (449 frente a 397) y 460 € de coste más en esas 1500 reservas (de 17.810 € a 18.270 €).
```

Cada **figura** lleva título que dice lo que se ve, ejes con nombre, número, pie y una mención en el texto. Para guardar el gráfico de subgrupos del paso 1, añadid esta línea justo antes de `plt.show()`:

```
fig.savefig("fig_subgrupos.png", dpi=150, bbox_inches="tight")
```

`fig.savefig` guarda la imagen con el nombre indicado, `dpi=150` le da calidad para imprimir y `bbox_inches="tight"` recorta los márgenes. Después descargadla desde el panel de archivos de Colab (icono de carpeta). Pie de ejemplo: «Figura 4. En el canal directo detectamos solo el 62 % de las cancelaciones (test de 1500 reservas; las barras de error son el margen del 95 %). Fuente: `04_evaluacion.ipynb`.»

7. **Comprobar la coherencia y declarar la IA (5 minutos).** Una persona que no escribió esas partes comprueba que lo mismo dice lo mismo en todos los sitios:

Dato	Dónde aparece	Tiene que ser igual en
AUC y recall del modelo	Cuaderno del paso 1	Model card (5), memoria (5), `auc_test` del demostrador y README
Cifras por subgrupo, antes y después	Cuaderno del paso 3	Model card (6), memoria (6) y `registro.csv`
Limitaciones y usos no permitidos	Model card (8)	`limitaciones` del demostrador, memoria (6 y 9) y `riesgos_eticos.md` (Parte D)
Nivel AI Act y datos personales	`riesgos_eticos.md` (Parte B)	Model card (7), README (apartado 7) y memoria (6)

   Si habéis usado una IA para redactar o programar, **declaradlo**: qué pedisteis, qué revisasteis y qué cambiasteis. Cada persona lo escribe en su hoja de autoevaluación (la frase «He usado IA para ……, y lo que he comprobado a mano es ……») y el equipo lo resume en «Referencias y licencias» de la memoria, que no tiene un apartado fijo para ello. Ejemplo: pedimos «Corrige la ortografía y la claridad de este párrafo sin cambiar el contenido» (sección 6); revisamos que 61,6 %, 79,2 % y +52 falsas alarmas seguían iguales; cambiamos dos frases demasiado largas.

Qué deberíais ver.

- **En el cuaderno:** las ocho líneas con `| canal = … |`; por ejemplo `| canal = directo (antes) | 388 | recall 61.6% (± 8.5) | -21.2 puntos |` y `| canal = directo (después) | 388 | recall 79.2% (± 7.1) | -6.8 puntos |`. Y el archivo `fig_subgrupos.png` en el panel de archivos.
- **En el repositorio:** `docs/model_card.md` sin ningún `_____`, con las tablas bien dibujadas en GitHub; `docs/riesgos_eticos.md` con la columna S16 del checklist, la Parte B justificada, al menos 5 riesgos y cada riesgo alto con estado y evidencia, y la Parte D rellena.
- **En el documento de la memoria:** las secciones 1 a 9 con texto (la 0 se escribe el día 3), figuras con pie y los `[PENDIENTE: …]` que queden localizados.

Hecho cuando: la model card está completa, la clasificación RGPD / AI Act está justificada con evidencia, cada riesgo alto tiene estado y evidencia, y la memoria tiene todos los apartados escritos aunque sea en borrador.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'antes' is not defined` (o `despues`, `pred_nueva`)	Colab se reinició o no se ejecutaron las celdas del paso 3.	Pulsad ▶ por orden desde la primera celda del cuaderno.
`KeyError: '±'`	La función `medir_subgrupos` no es la del paso 1 (le falta la columna ±).	Volved a copiar la celda del paso 1 y ejecutad de nuevo las tablas.
Una tabla se ve en GitHub como texto suelto, sin dibujar	Falta la fila de guiones bajo la cabecera, o una fila está partida en dos líneas, o hay una barra vertical dentro de una celda.	Editad con : cada fila en una sola línea y la fila de guiones justo debajo de la cabecera.
La model card ocupa más de 2 páginas	Frases largas o demasiadas filas.	Dejad solo los subgrupos con diferencia grande y casos suficientes; una frase por celda.
Un riesgo alto sin evidencia, o con «hemos cuidado…»	La evidencia tiene que poder comprobarse.	Escribid la ruta y el apartado (`sprint-16/docs/model_card.md`, apartado 8). Si no existe, es un ◐ o sigue `abierto`.
La misma cifra sale distinta en la model card y en la memoria	Se copió de versiones distintas del cuaderno.	Volved a sacarla del cuaderno ejecutado y usad la tabla del punto 7.
No aparece `fig_subgrupos.png` en el panel de archivos	La línea `fig.savefig` va después de `plt.show()`, o el panel no se ha refrescado.	Ponedla justo antes de `plt.show()`, ejecutad la celda y pulsad el icono de refrescar.

Paso 6 · Verificar en limpio y corregir (día 3, 80 minutos)

1. Una persona que no escribió el README lo sigue en un Colab nuevo o una carpeta limpia (60 minutos).
2. Mientras tanto, el resto cierra la memoria y la exporta a PDF.
3. Corregir lo que haya fallado (20 minutos).

En sencillo. «A mí me funciona» no es una prueba. Un proyecto está terminado cuando **otra persona, en otro sitio, lo hace funcionar leyendo solo el README**. Es como la receta que escribís para un amigo: no vale que a vosotros os salga el plato, tiene que salirle a él sin llamaros por teléfono. Por eso hoy una persona que no escribió el README ejecuta todo desde cero, apunta cada tropiezo con su mensaje exacto, y el equipo lo arregla. No hace falta saber programar: quien prueba solo obedece el README y copia los mensajes de error; arreglar suele ser añadir una línea al README o a `requirements.txt`. Mientras, el resto del equipo deja la memoria lista y la exporta a PDF.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Prueba en limpio	Ejecutar el proyecto en un sitio donde no hay nada vuestro instalado ni guardado, para ver si funciona solo con lo que hay en el repositorio.
ZIP del repositorio	Todo el repositorio en un solo archivo, que se descarga con **Code → Download ZIP**. Lo que no está subido, no está en el ZIP.
Ruta absoluta	Dirección de un archivo en *vuestro* ordenador (`/content/drive/MyDrive/datos.csv`, `C:\Users\Ana\…`). En otro ordenador no existe.
Dependencia	Una librería que el proyecto necesita (`seaborn`, `streamlit`). Si no está en `requirements.txt`, otra persona no la tiene.
Semilla	El número (42) que hace que un reparto al azar salga igual en cada ejecución. Sin semilla, las cifras cambian y no coinciden con la memoria.
Registro de fallos	Tabla donde se apunta cada tropiezo: qué se hacía, qué se esperaba y qué pasó (con el mensaje copiado).
Exportar a PDF	Guardar el documento en un formato que se ve igual en todos los ordenadores.

Cómo se hace

1. **Repartir los papeles (3 minutos).**

Papel	Quién	Qué hace
Verificador/a	Alguien que **no** escribió el README	Ejecuta todo siguiendo solo el README, en voz alta. Si lo prueba quien lo escribió, no vale: ya conoce los trucos que no están anotados
Observador/a	Quien escribió el README	**No ayuda ni explica.** Apunta cada fallo en el registro y ejecuta el punto 4
Memoria	El resto del equipo	Punto 5, a la vez

2. **Preparar el entorno limpio (10 minutos).** El/la verificador/a descarga el repositorio del equipo: en github.com, botón verde **Code → Download ZIP**. En Colab (colab.research.google.com → **Nuevo cuaderno**) abre el panel de archivos (icono de carpeta, a la izquierda), arrastra el ZIP y ejecuta esta celda cambiando el nombre por el de vuestro ZIP:

```
!unzip -q ifcd107-equipo-NOMBRE-main.zip
%cd ifcd107-equipo-NOMBRE-main
!ls
!pip install -r requirements.txt
```

   Qué hace cada línea:

   - `!unzip -q …` descomprime el ZIP (el `!` le dice a Colab que es una orden de terminal, no de Python). La carpeta que aparece se llama como el repositorio más `-main`.
   - `%cd …` entra en esa carpeta; a partir de aquí las rutas del README funcionan.
   - `!ls` lista lo que hay. Con un repositorio de ejemplo salen `README.md`, `requirements.txt`, `sprint-14`, `sprint-15` y `sprint-16`. Si falta algo, **no se subió**.
   - `!pip install -r requirements.txt` instala la lista del proyecto.

   Si el README empieza con `git clone`, saltadlo: ya tenéis el contenido.

3. **Seguir el README al pie de la letra (40 minutos).** Quien verifica va leyendo el README y ejecutando **exactamente** lo que dice, en orden: datos, cuadernos de arriba abajo (**Entorno de ejecución → Ejecutar todo**), demostrador (`streamlit run app.py` no se abre en Colab: se hace en un ordenador, como en el paso 4) y pruebas. Reglas:
   - Si tiene que **adivinar** algo, es un fallo del README, aunque al final lo consiga.
   - Si hay un error, **copia el mensaje entero**; no lo resume.
   - Compara cada cifra del cuaderno con la de la memoria y el README (con tres decimales). Tienen que ser iguales.

   El/la observador/a apunta cada fallo en una tabla como esta (ejemplos):

Hora	Qué se hacía (línea del README)	Qué se esperaba	Qué pasó (mensaje copiado)	Arreglo	Quién
10:20	Ejecutar `04_evaluacion.ipynb`	Que cargase los datos	`FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/MyDrive/reservas_turisdata.csv'`	Ruta relativa o `dato(...)`	Ana
10:45	Lanzar el demostrador	Que se abriese la app	`FileNotFoundError: Falta el modelo. Ejecuta primero el notebook S16_01…`	Subir `modelo/` o explicar en el README cómo generarlo	Luis

4. **Buscar lo que siempre falla (7 minutos).** Lo hace el/la observador/a (que no puede ayudar a quien verifica). Desde la carpeta del repositorio descomprimido, estas dos revisiones. **Primera:** rutas de vuestro ordenador dentro de los archivos.

```
from pathlib import Path

RUTAS_SOSPECHOSAS = ["/content/", "/home/", "/Users/", "C:\\Users", "C:/Users", "drive/MyDrive"]

for archivo in sorted(Path(".").rglob("*")):
    if archivo.suffix in (".py", ".ipynb", ".md", ".txt", ".json") and ".venv" not in archivo.parts:
        texto = archivo.read_text(encoding="utf-8", errors="ignore")
        for marca in RUTAS_SOSPECHOSAS:
            if marca in texto:
                print(f"{archivo}: contiene '{marca}'")
```

   Qué hace cada línea:
   - `RUTAS_SOSPECHOSAS` es la lista de trozos de dirección que delatan un ordenador concreto. Podéis añadir los vuestros.
   - `Path(".").rglob("*")` recorre **todas** las carpetas desde donde estáis. El `if` se queda con los archivos de texto y se salta `.venv`.
   - `archivo.read_text(…)` lee el archivo y el bucle `for marca` escribe una línea por cada marca encontrada.

   Con un repositorio de ejemplo sale `sprint-16/notebooks/04_evaluacion.ipynb: contiene '/content/'`. Cada línea es una ruta que hay que cambiar por una relativa. Un `/content/` legítimo (una ruta de Colab en un comentario) se puede dejar si el README lo explica. **Segunda:** las librerías que usan vuestros cuadernos, para compararlas con `requirements.txt`.

```
import json
import re
import sys
from pathlib import Path

librerias = set()
for cuaderno in Path(".").rglob("*.ipynb"):
    for celda in json.loads(cuaderno.read_text(encoding="utf-8"))["cells"]:
        if celda["cell_type"] == "code":
            for linea in "".join(celda["source"]).splitlines():
                m = re.match(r"\s*(?:import|from)\s+([A-Za-z0-9_]+)", linea)
                if m:
                    librerias.add(m.group(1))
print(sorted(librerias - set(sys.stdlib_module_names)))
```

   Escribe la lista de librerías que importan los cuadernos, sin las que ya trae Python (`sys.stdlib_module_names`). Con los cuadernos `S13_02` y `S16_01` sale `['IPython', 'api', 'fastapi', 'ipywidgets', 'joblib', 'logica', 'matplotlib', 'numpy', 'pandas', 'pydantic', 'seaborn', 'sklearn', 'streamlit']`. `api` y `logica` son archivos del propio demostrador, no librerías, e `IPython` ya viene con Jupyter y Colab. Comprobad que cada una de las demás está en `requirements.txt`; `sklearn` se instala como `scikit-learn`.

5. **Memoria a limpio y PDF (el resto del equipo, 60 minutos).** Por este orden:
   - **Pendientes.** Buscad `PENDIENTE` con `Ctrl + F` y resolvedlos uno a uno.
   - **Sección 0, resumen ejecutivo.** Se escribe **la última**, con las cuatro viñetas del esqueleto y vuestras cifras. Ejemplo con TurisData:

     ```
     Problema y decisión: cada cancelación sin aviso deja una habitación vacía; quien gestiona la disponibilidad decide si pide depósito.
     Qué hicimos: comparamos modelos con validación cruzada y evaluamos el candidato una sola vez en el test.
     Resultado: AUC 0,744 (95 %: 0,719–0,767) frente a 0,50 del baseline; recall 82,8 %. Con los costes supuestos, 17.810 € frente a 34.400 € sin modelo en 1500 reservas de prueba.
     Recomendación y límites: usarlo para orientar, con una persona que decida; umbral propio en el canal directo; no usarlo para decidir sobre personas. Datos sintéticos de un solo alojamiento.
     ```

   - **Portada, índice y «Referencias y licencias».** En las referencias: cada fuente de datos con la cita que dejasteis en `plan_datos_licencias.md` (apartado 7: fuente, autor, título, año, versión, enlace, licencia), las librerías con sus versiones y la declaración del uso de IA del paso 5.
   - **Anexos A a G** del esqueleto (ficha de proyecto, plan de datos, registro de experimentos, model card, registro de riesgos, capturas del tablero y retros, enlace al repositorio y a `v1.0-entrega`; la dirección de la etiqueta acaba en `/releases/tag/v1.0-entrega`, así que podéis escribirla ya aunque la etiqueta se cree en el paso 8).
   - **Lista de calidad** del final del esqueleto, con una persona que **no** escribió esa sección: cada figura con título que dice lo que se ve, ejes con nombre y citada en el texto; cifras iguales que en el README y el registro; cada afirmación importante con evidencia; sin párrafos genéricos sobre «qué es la IA».
   - **Contar las páginas.** Las que cuentan son las de las secciones 0 a 9; no cuentan portada, índice, referencias ni anexos. Ejemplo de cuenta (no es vuestro número):

Parte	Páginas	¿Cuenta?
Portada e índice	2	No
Secciones 0 a 9	13	**Sí** (tiene que dar entre 10 y 15)
Referencias y licencias	1	No
Anexos A a G	4	No
Total del PDF	20	

     Si pasáis de 15, recortad texto repetido y figuras grandes antes de quitar contenido (el esqueleto da el presupuesto de cada sección). Si no llegáis a 10, falta contenido en alguna sección.
   - **Exportar.** En Google Docs: **Archivo → Descargar → Documento PDF**; en Word: **Archivo → Guardar como → PDF**. Nombre: `memoria_tecnica.pdf`. Abrid el PDF y mirad cada página: figuras sin cortar, tablas dentro del margen, números de página. Subidlo a `sprint-16/docs/` con **Add file → Upload files**.
6. **Corregir y repetir (20 minutos).** Con el registro de fallos delante, arreglad uno a uno y **repetid solo la parte que falló**:

Fallo	Arreglo
Falta una librería	`pip show NOMBRE` y copiad la línea `Version:` a `requirements.txt` como `NOMBRE==x.y.z`
Ruta absoluta	Una ruta relativa desde la carpeta del archivo, o `dato("archivo.csv")` si el dato es del curso
Falta un archivo (datos, `modelo/`)	Subirlo si pesa poco y la licencia lo permite; si no, un script de descarga y una frase en el README
Falta un paso en el README	Añadidlo con la orden exacta y qué se debe ver
Las cifras cambian de una ejecución a otra	Fijad `SEMILLA = 42` y usadla en el reparto de datos y en el modelo; ejecutad de arriba abajo

   Cuando algo cambie, subidlo (**Add file → Upload files** o el lápiz), y que quien verifica repita ese fragmento. Lo que no se pueda arreglar hoy, apuntadlo en la model card y en la memoria como limitación.

Qué deberíais ver.

- **El registro de fallos:** corto, con cada fila resuelta y con el mensaje de error copiado.

- **En el entorno limpio:** `pip install -r requirements.txt` termina sin errores, los cuadernos se ejecutan de arriba abajo con las mismas cifras que la memoria y, en un ordenador, `pytest -q` da `8 passed` (el número de vuestras pruebas) y la app se abre en `http://localhost:8501`.
- **Los dos buscadores:** el de rutas sin ninguna línea (o solo las que justificáis) y el de librerías con todas dentro de `requirements.txt`.
- **En `sprint-16/docs/`:** `memoria_tecnica.pdf` con 10 a 15 páginas contadas.

Hecho cuando: otra persona ha reproducido el proyecto con solo el README, lo que falló está corregido y existe `docs/memoria_tecnica.pdf`.

Si algo falla

Veis…	Qué significa	Qué hacer
`ModuleNotFoundError: No module named 'seaborn'` (o `scipy`, `streamlit`…)	La librería estaba instalada en vuestro entorno, pero no está en `requirements.txt`.	Añadidla con su versión (`pip show seaborn`) y volved a instalar. El buscador de librerías del punto 4 las encuentra todas.
`FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/MyDrive/…'`	Una ruta absoluta de otro ordenador.	Cambiadla por una ruta relativa; el buscador de rutas del punto 4 localiza el resto.
`ERROR: Could not find a version that satisfies the requirement numpy==9.9.9` y debajo `ERROR: No matching distribution found for numpy==9.9.9`	La versión de `requirements.txt` no existe o está mal escrita.	Copiad la línea `Version:` de `pip show` tal cual, sin cambiar ninguna cifra.
`FileNotFoundError: Falta el modelo. Ejecuta primero el notebook S16_01…`	`demostrador/modelo/` no está en el repositorio.	Subid la carpeta `modelo/` (pesa poco) o explicad en el README cómo generarla.
Las cifras del cuaderno no coinciden con las de la memoria	La semilla no está fijada, o la memoria se escribió con otra versión del cuaderno.	Fijad `SEMILLA`, ejecutad de arriba abajo y corregid la memoria con las cifras nuevas **en todos los sitios** (paso 5, punto 7).
El PDF tiene más de 15 páginas contadas (o menos de 10)	Texto repetido, figuras muy grandes, o secciones demasiado cortas.	Recortad o ampliad según el presupuesto de páginas del esqueleto y volved a exportar.
`[Errno 2] No such file or directory: 'ifcd107-equipo-…'` al ejecutar `%cd`	El nombre de la carpeta no es el que escribisteis.	Ejecutad `!ls` y copiad el nombre exacto que sale.

Paso 7 · Plan B, riesgos cerrados, lista de congelación y coevaluación (día 3, 65 minutos)

1. Grabar el vídeo plan B del demostrador (30 minutos).
2. Cerrar el registro de riesgos y pasar la lista de congelación (35 minutos).
3. Rellenar vuestra coevaluación individual (anexo B).

En sencillo. Antes de dar una obra por terminada se hacen tres cosas. Un **seguro**: un vídeo de 90 segundos de la demo funcionando, por si el día de la defensa falla internet o la app. Una **revisión final con lista**, como la de un piloto antes de despegar: casilla a casilla, sin fiarse de la memoria. Y una **valoración honesta del equipo**, en privado. No hace falta programar: se graba la pantalla con lo que trae el ordenador, se marcan casillas y se escribe sobre lo que cada persona ha hecho. Un pequeño programa de unas líneas os ayuda a buscar claves y datos personales olvidados en el repositorio.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Plan B	Lo que usáis si falla el plan A. Aquí, un vídeo del demostrador funcionando (`docs/demo.mp4`).
Guion	Lista corta de lo que se ve y se dice en cada tramo del vídeo, para no improvisar.
Congelación	A las 14:30 del 10/12 lo que hay en la rama principal es lo que se evalúa; después no se cambia código ni memoria.
Lista de congelación	`checklist_congelacion.md`: seis bloques (A a F) de casillas que se marcan **con evidencia**.
Clave (API key)	Contraseña de un servicio. Va en los **Secretos** de Colab, nunca escrita en un archivo del repositorio.

Palabra	Qué significa, en sencillo
Dato personal	Cualquier dato que identifique a una persona (nombre, correo, teléfono, DNI). No puede estar en el repositorio.
Coevaluación (anexo B)	Cada persona puntúa en privado a todo el equipo, incluida ella. Se entrega al docente, no al repositorio.

Cómo se hace

1. **Repartir (2 minutos).** Tres tareas a la vez: el **vídeo** (la persona D y otra), la **lista de congelación y la búsqueda de claves** (la persona que verificó en el paso 6 y otra) y la **coevaluación** (cada persona, sola, al acabar su parte).
2. **Grabar el vídeo plan B (30 minutos).** Preparad la demo: `streamlit run app.py` en la carpeta `demostrador/`, el navegador con el zoom al 125 % para que se lea, sin pestañas personales abiertas ni avisos que puedan saltar. Elegid las tres entradas **antes** de grabar. Duración: entre 1 y 2 minutos, unos 90 segundos. Guion con las entradas del paso 4:

Segundos	Qué se ve	Qué se dice
0–10	La app abierta	«Esto es el demostrador de riesgo de cancelación de TurisData. Usa datos sintéticos.»
10–35	Una entrada típica y **Calcular riesgo**	«Escribo una reserva con 120 días de antelación y tarifa flexible sin depósito. Da un 81 %, riesgo alto, frente a una media del 46 %.»
35–55	Otra entrada distinta	«Con poca antelación, depósito total y tarifa no reembolsable, el riesgo baja al 4 %.»
55–75	Una entrada **errónea a propósito**	«Pongo una antelación de 5000 días. La app no se rompe y dice que tiene que estar entre 1 y 400.»
75–90	El cuadro azul del aviso	«Cada resultado lleva este aviso: el AUC del modelo y para qué no debe usarse.»

   Para grabar, lo que trae el ordenador: en **Windows**, la barra de juegos (`Windows + Alt + R` empieza y acaba la grabación de la ventana activa); en **Mac**, `Cmd + Mayús + 5`; o la grabación de una videollamada compartiendo pantalla. Después:

   - **Vedlo entero, con sonido.** ¿Se lee? ¿Aparece la entrada errónea? ¿Hay algo personal en pantalla?
   - **Peso.** Si es menor de 25 MB, subidlo a `sprint-16/docs/demo.mp4` (**Add file → Upload files**). Si pesa más, regrabad una ventana en vez de la pantalla completa o más corto; o subidlo a un sitio estable (un enlace que abra sin iniciar sesión) y poned el enlace en el README.
   - **Formato.** Si vuestro programa no guarda `.mp4` (Mac guarda `.mov`), subidlo con su extensión y cambiad el nombre en el README y en la lista; o preguntad al docente cómo convertirlo.
3. **Pasar la lista de congelación (25 minutos).** Una persona **distinta a la que escribió el README** abre `checklist_congelacion.md` y la copia al tablero o la imprime. Va casilla a casilla y marca solo lo que **ha visto con sus ojos**. Dónde mirar:

Bloque	Cómo se comprueba
A · Repositorio	El icono del reloj (*Commits*): salen las aportaciones de **todas** las personas. `README.md` con todos sus apartados, `requirements.txt` con versiones y versión de Python. Todo subido
B · Reproducibilidad	El registro de fallos del paso 6, con todo resuelto
C · Demostrador	Se lanza con un solo comando; las 5 entradas del paso 4; aviso de incertidumbre en cada resultado; `docs/demo.mp4` o su enlace
D · Evaluación e IA responsable	Test usado una sola vez; subgrupos y mitigación medida; `docs/riesgos_eticos.md` con la Parte D rellena (fecha y firmas incluidas); `docs/model_card.md` sin `_____`
E · Memoria	`docs/memoria_tecnica.pdf` con 10 a 15 páginas contadas y las cifras de la tabla del paso 5 iguales en todos los sitios
F · Equipo	Capturas del tablero (backlog, planning, review y retro de los sprints 14, 15 y 16) en `docs/scrum/`; la retro con una acción concreta; coevaluación hecha

   El registro de riesgos se cerró en el paso 5. Aquí solo se comprueba, y si el paso 6 destapó algo nuevo (por ejemplo, un modelo que no cabe en el repositorio) se añade como riesgo con su estado. **Cada casilla sin marcar** se arregla ahora, antes de las 14:30, o se apunta en el tablero con responsable y hora. Tras las 14:30 no se cambia código ni memoria.

4. **Buscar claves y datos personales (10 minutos).** Quien pasa la lista descomprime el ZIP del repositorio en Colab y entra en su carpeta con `%cd`, como en el paso 6. Después ejecuta esto en una celda:

```
import re
from pathlib import Path

PATRONES = {
    "posible clave": r"(?:AIza|gsk_|sk-)[0-9A-Za-z_\-]{20,}",
    "clave escrita en el código": r"api_key\s*=\s*[\"'][^\"']{12,}[\"']",
    "correo": r"[\w.+-]+@[\w-]+\.[\w.-]+",
    "teléfono": r"\b[6-9]\d{8}\b",
    "DNI": r"\b\d{8}[A-Za-z]\b",
}

for archivo in sorted(Path(".").rglob("*")):
    if archivo.is_file() and ".venv" not in archivo.parts:
        if archivo.name.startswith(".env"):
            print(f"{archivo}: es un archivo de claves, no debe subirse")
        if archivo.suffix in (".py", ".ipynb", ".md", ".txt", ".json", ".csv"):
            texto = archivo.read_text(encoding="utf-8", errors="ignore")
            for nombre, patron in PATRONES.items():
                for hallazgo in re.findall(patron, texto):
                    print(f"{archivo}: {nombre} → {hallazgo[:6]}…")
```

   Qué hace cada parte:

   - `PATRONES` son «formas» de texto que delatan algo: una clave que empieza por `AIza` (Google), `gsk_` (Groq) o `sk-`, un `api_key = "…"` escrito en el código, un correo, un teléfono de 9 cifras o un DNI. Cada forma es una *expresión regular*; no hace falta entenderlas, solo ejecutarlas.
   - El primer `if` avisa si hay un archivo `.env`, que es donde algunos programas guardan claves.
   - El segundo `if` lee los archivos de texto y `re.findall` busca cada forma. Solo se escriben los **6 primeros caracteres** de cada hallazgo, para no volver a mostrar una clave entera.

   Con un repositorio de ejemplo sale, por ejemplo, `sprint-12/asistente.ipynb: posible clave → AIzaSy…`. Un `api_key=userdata.get("GEMINI_API_KEY")` **no** sale: es la forma correcta de usar los Secretos de Colab (guía de acceso a modelos). Revisad cada hallazgo: un correo del README del equipo también es un dato personal. Si encontráis una **clave**, borrarla del archivo no basta, porque queda en el historial de cambios: creadla de nuevo en la web del servicio, desactivad la anterior y avisad al docente. Si es un **dato personal**, quitadlo del archivo y avisad al docente.

5. **Coevaluación individual, anexo B (15 minutos).** La hace **cada persona sola** y no se comenta con el resto. Copiad `plantilla_coevaluacion.md` a un documento **vuestro** (no al repositorio) y rellenadla así:
   - **Puntuaciones (apartado 2).** De 1 a 4 a cada persona, **incluyéndoos a vosotros mismos**, en cinco dimensiones: contribución técnica, cumplimiento de tareas, comunicación, colaboración y fiabilidad. El 3 es «cumplió lo esperado». Un **1 o un 4 llevan un comentario obligatorio** con un hecho concreto.
   - **Reparto (apartado 3).** El esfuerzo global de cada persona en porcentaje; la suma da **100 %**.
   - **Preguntas abiertas (apartado 4).** Cinco respuestas de dos o tres frases con ejemplos: qué ha funcionado mejor; qué cambiaríais y qué acción de la retro sí se cumplió y cuál no; qué habéis aprendido y enseñado; algo que el docente deba saber; si rotasteis los roles de verdad.
   - **Firma.** Vuestro nombre.

   Ejemplo de un equipo de tres (Ana rellena el suyo):

Persona	Contribución técnica	Cumplimiento de tareas	Comunicación	Colaboración	Fiabilidad	Comentario (obligatorio si hay 1 o 4)
Yo mismo/a (Ana)	3	3	3	3	3	
Luis	2	2	3	3	3	
Marta	3	3	4	4	3	Comunicación y colaboración: el 9/12 vio que el recall no coincidía entre la memoria y el README, avisó en el grupo y lo corrigió en los tres sitios

   Reparto del ejemplo: Yo 30 % · Luis 30 % · Marta 40 % = 100 %. La plantilla pide **hechos, no etiquetas**. Mal: «Luis es vago». Bien: «Luis, cumplimiento de tareas (2): el apartado 8 de la model card se entregó un día más tarde de lo acordado y lo terminó otra persona». Distinguid lo que hizo de cómo os sentó y recordad el objetivo: mejorar el trabajo del equipo, no castigar. Se entrega **al docente**, en la forma que él indique; **no se sube al repositorio**. No la confundáis con la hoja `autoevaluacion_nombre.md`, que sí se sube (paso 8).

Qué deberíais ver.

- **El vídeo:** `docs/demo.mp4` en el repositorio, o su enlace en el README, de unos 90 segundos, con una entrada errónea que no rompe la app.
- **La búsqueda de claves:** ninguna línea, o solo hallazgos revisados y resueltos. Con el demostrador del paso 4 no sale ninguna.
- **La lista de congelación:** las seis secciones con todas las casillas marcadas, o con lo pendiente apuntado (qué, quién y a qué hora).
- **La coevaluación:** cada persona con la suya hecha, con comentario en cada 1 o 4 y reparto que suma 100 %.

Hecho cuando: el vídeo plan B existe, el registro de riesgos está cerrado, la lista de congelación está pasada y todo el equipo ha entregado la coevaluación.

Si algo falla

Veis…	Qué significa	Qué hacer
El archivo del vídeo no se sube	Por el navegador, GitHub admite 25 MB por archivo.	Regrabad más corto o una ventana en lugar de la pantalla completa; si sigue pesando, sitio estable y enlace en el README.
El vídeo se corta o no se oye	La grabación empezó en otra ventana o sin micrófono.	Repetidlo: son 90 segundos. Probad antes 10 segundos.
El buscador marca `posible clave → AIzaSy…`	Hay una clave escrita en un archivo.	Creadla otra vez en la web del servicio, desactivad la vieja, guardad la nueva en los Secretos de Colab y avisad al docente.
El buscador marca un `correo` o un `teléfono` en el README	Hay un dato personal.	Quitadlo (una dirección genérica del equipo, o ninguna) y avisad al docente.
En *Commits* solo aparece una persona	El resto no ha subido nada con su cuenta.	Que cada persona suba **ahora** lo suyo (su autoevaluación vale) desde su cuenta; no se arregla después de las 14:30.
Una casilla de la lista no se puede marcar	Falta algo o no hay evidencia.	Arreglarlo si hay tiempo; si no, apuntad qué falta, quién y cuándo. Una casilla marcada sin mirar es peor que una sin marcar.
La coevaluación no da 100 %	Los porcentajes del apartado 3 no suman bien.	Sumadlos y ajustad antes de entregarla.

Paso 8 · Revisión con la rúbrica, review, retro y congelación (día 3, 55 minutos)

1. Revisar el trabajo con la rúbrica y anotar el resultado en el README.
2. Hacer la review (unos 5 minutos por equipo) y la retro.
3. Subir todo a `sprint-16/`, crear la etiqueta `v1.0-entrega` y subir las autoevaluaciones, **antes de las 14:30**.

En sencillo. Es el último paso, y el que más se parece a entregar un trabajo de verdad. Primero, **una persona que no escribió esa parte lo corrige con la rúbrica**, como lo corregiría el docente, y anota la nota con una frase de prueba. Luego se enseña el trabajo (la *review*) y el equipo mira cómo ha trabajado (la *retro*). Por último se sube todo al repositorio y se pone una **etiqueta**: es como cerrar el sobre y firmar la hora, para que quede claro qué versión se evalúa. No hace falta Git ni programar: casi todo se hace desde el navegador, con los botones de GitHub.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Rúbrica	La tabla de 4 criterios por 4 niveles con la que se puntúa el reto. El equipo la usa para revisarse y el docente para puntuar.
Frase de prueba	La razón comprobable de un nivel: «3 · el README funciona en un Colab nuevo, pero falta la versión de Python». Sin ella, el nivel no vale.
Review	Enseñar el trabajo terminado al docente y a la clase (unos 5 minutos por equipo).
Retro	Reunión del equipo para mejorar la forma de trabajar: qué mantener, qué cambiar y qué acción concreta.
Etiqueta (*tag*, `v1.0-entrega`)	Marca con nombre y hora que apunta a una versión del repositorio. Lo que tiene a las 14:30 es lo que se evalúa.
Release	La página de GitHub desde la que se crea la etiqueta, con un título y una descripción.
Commit	Guardar un cambio en GitHub con un mensaje. Cada commit lleva el nombre de quien lo hace.

Cómo se hace

1. **Revisar con la rúbrica (20 minutos).** Lo hace una persona que **no escribió esa parte** (una el repositorio y el demostrador; otra, la memoria y la model card). Abrid entrega y rúbrica, apartado 3, y los cinco pasos de «Cómo se revisa». Para cada uno de los cuatro criterios: leed las columnas de izquierda a derecha y parad en la primera cuyo texto **no se cumple entero**; vuestro nivel es el de la columna anterior (si lo de un nivel se cumple a medias, es el de abajo). Escribid **una frase de prueba** con algo que habéis comprobado. Después abrid `README.md`, lápiz , y sustituid el texto de la sección **«Revisión con la rúbrica»** (el final del README) por algo así:

```
## Revisión con la rúbrica

*Sprint 16. Revisó: Luis (no escribió el README) · 10/12/2026.*

| Criterio | Nivel (1-4) | Frase de prueba |
|---|---|---|
| Corrección técnica | 3 | Test usado una vez (AUC 0,744); recall del canal directo 61,6 % → 79,2 % medido antes y después; `pytest -q` da 8 passed |
| Reproducibilidad | 3 | El README funciona en un Colab nuevo; versiones fijas; falta indicar el tiempo total. Etiqueta pendiente de crear |
| Análisis y comunicación al cliente | 3 | Memoria de 13 páginas; cifras iguales en memoria, README y model card; dice qué no hacer con el modelo |
| IA responsable | 3 | Clasificación justificada (riesgo mínimo, sin datos personales); R-01 mitigado con evidencia; uso de IA declarado |

**Lo primero que mejoraríamos:** indicar el tiempo total de reproducción en el README.
```

   Qué hay que respetar: la etiqueta se crea **después** de esta revisión, así que en Reproducibilidad anotad «etiqueta pendiente» y comprobadla en el punto 5. Lo fácil de arreglar, arreglarlo ya; lo demás, en «Lo primero que mejoraríamos». Nada de niveles sin frase de prueba. Si dos personas editan el README a la vez, GitHub avisa de un conflicto (mirad la tabla de abajo).

2. **Prepararse para la review (5 minutos).** Tenéis unos 5 minutos: **4 de demostración y 1 con una diapositiva**. Repartid los papeles: quien enseña la app, quien cuenta el sesgo, quien cuenta los riesgos, quien apunta el tiempo y el resultado de la prueba del docente. Guion:

Minuto	Qué se hace
0–1	Qué es el proyecto y qué decide el cliente con él
1–2:30	Demostrador: una entrada típica y su aviso de incertidumbre
2:30–4	Una entrada errónea a propósito y **la entrada difícil que propone el docente**, en directo
4–5	Una diapositiva: tabla de sesgo y mitigación (antes y después, por subgrupo) y los riesgos altos con su estado

   Si la app falla con la entrada del docente, no os justifiquéis: abrid el vídeo plan B (`docs/demo.mp4`), contad qué ha pasado y apuntadlo como limitación. Durante las reviews de los demás equipos, tomad notas para la **Parte 2 de vuestra hoja de autoevaluación y coevaluación** (punto 6).

3. **La retro (10 minutos, el equipo).** Una cosa que **mantener**, una que **cambiar** y una **acción concreta** (qué, quién y para cuándo). Revisad también la acción de la retro anterior: ¿se hizo y qué cambió? Apuntadlo en el tablero y haced una captura para `docs/scrum/`. Ejemplo:

Mantener	Cambiar	Acción (qué · quién · cuándo)	Acción de la retro anterior
La revisión cruzada de cifras antes de congelar	Dejamos la memoria hasta el día 2 y fue muy justo	Ensayar la defensa completa, con cronómetro · Marta · antes de la defensa	Cumplida: revisar las cifras entre dos personas evitó un recall distinto en la memoria y el README

4. **Subir los ficheros (10 minutos).** Con Cómo entregar en GitHub: abrid el repositorio del equipo; **Add file → Create new file** y escribid la ruta completa (cada barra crea una carpeta), o **Add file → Upload files** y arrastrad los archivos (o la carpeta entera); mensaje corto y **Commit changes**. Nombres en minúsculas y sin espacios. **Que suban personas distintas, cada una desde su cuenta**: el historial es la prueba de que trabajó todo el equipo. Reparto de ejemplo con tres personas:

Quién	Qué sube	Dónde
Ana	`README.md` (y la raíz al día) · `04_evaluacion.ipynb`	`sprint-16/` · `sprint-16/notebooks/`
Luis	`logica.py` · `app.py` · `requirements.txt` · `tests/test_logica.py` · `modelo/` (si pesa poco)	`sprint-16/demostrador/` (y `…/demostrador/tests/`)
Marta	`riesgos_eticos.md` · `model_card.md` · `memoria_tecnica.pdf` · `demo.mp4` (si pesa menos de 25 MB)	`sprint-16/docs/`

   El cuaderno se descarga desde Colab con **Archivo → Descargar → Descargar .ipynb**; ejecutadlo antes de arriba abajo para que se guarden las salidas. Si ya subisteis algo en los pasos anteriores, no lo repitáis. Comprobad con el reloj de *Commits* que sale **cada persona**.

5. **Crear la etiqueta (5 minutos).** Es lo **último**, cuando todo lo anterior está subido. En la página principal del repositorio, a la derecha, **Releases → Create a new release**; en *Choose a tag* escribid `v1.0-entrega` y elegid *Create new tag*; el destino es `main`; poned un título (por ejemplo «Entrega final») y pulsad **Publish release**. Los nombres de los botones pueden cambiar un poco. Con Git en un terminal sería `git tag v1.0-entrega` y `git push origin v1.0-entrega`. Comprobad que la etiqueta aparece en la lista de *Releases* y que se publicó **antes de las 14:30**. Después de esa hora no se cambia código ni memoria; un fallo crítico del demostrador solo se corrige con permiso del docente y dejándolo documentado.

6. **Autoevaluación y coevaluación (5 minutos por persona, en paralelo).** Cada persona, con su propia cuenta, sube su hoja: copia el texto de `autoevaluacion_y_coevaluacion.md` (**Raw** → `Ctrl + A` → `Ctrl + C`), **Add file → Create new file** en `sprint-16/` con el nombre `autoevaluacion_marta.md` (vuestro nombre, minúsculas, sin tildes ni espacios), pega, rellena y **Commit changes**. Rellenad:

   - **Parte 1 (autoevaluación).** Contestad las nueve preguntas con Sí / No / A medias, puntuad de 1 a 4 los cuatro criterios con **una frase** y completad «He usado IA para ……, y lo que he comprobado a mano es ……».
   - **Parte 2 (coevaluación del trabajo revisado).** Nombrad el trabajo del equipo que os ha tocado, marcad , o — en cada entregable y puntuad los cuatro criterios con un comentario. Ejemplo de fila: *Demostrador* · · *«Se abrió con un comando y la entrada errónea dio un mensaje claro»*. Valorad el **trabajo**, no a las personas.

   No es la coevaluación confidencial (anexo B) del paso 7: esa va al docente, y esta hoja sí se sube al repositorio.

Qué deberíais ver. En `sprint-16/` los 9 ficheros del equipo (`README.md`, `notebooks/04_evaluacion.ipynb`, `demostrador/logica.py`, `app.py`, `tests/test_logica.py`, `requirements.txt`, y en `docs/` `riesgos_eticos.md`, `model_card.md` y `memoria_tecnica.pdf`) y una hoja `autoevaluacion_nombre.md` por persona (**9 + N**; un equipo de 3 sube 12).

En el historial, commits de todas las personas. En el README, la sección «Revisión con la rúbrica» con cuatro niveles y su frase de prueba. En *Releases*, `v1.0-entrega` con una hora anterior a las 14:30.

Hecho cuando: la sección «Revisión con la rúbrica» del README está rellena, todo está subido, la etiqueta existe antes de las 14:30 y cada persona ha subido su autoevaluación y coevaluación.

Si algo falla

Veis...	Qué significa	Qué hacer
No aparece **Add file**	No sois colaboradores o no habéis aceptado la invitación.	Aceptad la invitación (correo o github.com/notifications) o pedidla a quien creó el repositorio.
El archivo quedó en la raíz, no en `sprint-16/`	Se subió sin la ruta de la carpeta.	Abridlo, lápiz , y escribid delante del nombre `sprint-16/docs/` (la ruta que toque); **Commit changes** lo mueve.
Subisteis algo y no aparece	No se pulsó el botón verde **Commit changes**.	Subidlo otra vez y pulsadlo: sin él no se guarda nada.
Aviso de conflicto al guardar el README	Dos personas lo editaron a la vez.	Recargad la página y repetid vuestro cambio sobre la versión actual.
No encontráis **Releases** o **Create a new release**	Los botones de GitHub cambian de sitio, o vuestra cuenta no puede escribir en el repositorio.	Buscad *Releases* en la columna derecha de la página principal. Si no hay forma, que lo haga otra persona del equipo o avisad al docente.
En `sprint-16/` no salen 9 + N ficheros	Falta algún archivo o alguna hoja de autoevaluación.	Comparad con la tabla del punto 4 y con la lista de «Qué deberíais ver»; cada persona comprueba el suyo.
Son las 14:30 y la etiqueta no está	Se pasó la hora de congelación.	Avisad al docente **ya**, explicad qué falta y no cambiéis nada más.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **escribe con seguridad cosas que pueden ser falsas** y puede cambiar una cifra sin avisar. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una parte, tenéis que saber explicarla.
- **La memoria es vuestra:** la IA no la escribe por vosotros. Si la usáis para redactar o corregir, revisadla entera y **declaradlo en la memoria**.
- **Las cifras mandan:** después de que la IA toque un texto, comprobad que no ha cambiado ninguna.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender sesgos	«Explícame con un ejemplo qué significa que un modelo tenga peor rendimiento en un subgrupo y qué medidas existen»	Que podéis aplicar una medida **y medirla**
Crear la app	«Ayúdame a añadir validación de entradas a esta app de Streamlit: *[código]*»	Que **probáis** entradas absurdas
Estructurar la memoria	«Dame un guion de las secciones de una memoria técnica de un proyecto de IA»	Que cada sección la redactáis vosotros con **vuestros resultados**
Revisar la redacción	«Corrige la ortografía y la claridad de este párrafo sin cambiar el contenido»	Que no se ha cambiado ninguna cifra

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Dejar la memoria para el último día	«Total, ya está hecho»	Borrador el día 2 (9/12) y revisión el día 3 (10/12)
El demostrador solo funciona en el ordenador de una persona	Dependencias sin documentar	`requirements.txt` con versiones y prueba en un entorno limpio
Mitigación que se aplica pero no se mide	Se piensa que basta con hacerla	Comparad las métricas **antes y después, por subgrupo**
Subir cambios después de las 14:30	Se siguen retocando cosas	Crear la etiqueta antes y no tocar nada después

Error	Por qué ocurre	Cómo evitarlo
Coevaluación sin entregar	Se olvida	Cada persona la entrega; cuenta para la nota individual
Memoria con más de 15 páginas o con fragmentos copiados	Se llena de relleno	Respetad el presupuesto de páginas; los anexos no cuentan; escribid con vuestras palabras
Cifras distintas en la memoria y en el README	Se copian de versiones distintas	Una revisión final de coherencia antes de congelar
Solo una persona hace commits	«El resto no sabe Git»	Reparto real: vale que cada persona suba un fichero desde la web

Ayuda 3 · Si os atascáis

1. **El demostrador no arranca en otro ordenador:** faltan dependencias. Generad `requirements.txt` con `pip freeze` (o a mano), recortadlo a lo que usáis y probad en un entorno nuevo.
2. **No sabéis qué sesgo medir:** mirad las columnas con categorías (país, canal, tipo) y calculad la métrica para cada valor.
3. **Falta tiempo:** priorizad lo obligatorio (R1–R11 y la lista de congelación). Es mejor una memoria honesta con límites que una incompleta. Preguntad al equipo y, después, al docente.